Apostila 4 — Econometria com Python (Teoria Completa)

Teoria, demonstrações por simulação e diagnóstico, com foco em economia

Sumário

Apostila 4 — Econometria com Python (Teoria Completa) 1
Como Usar Esta Apostila 4
Capítulo 1 — O que é Econometria 4
1.1. Um pouco de história 4
1.2. Econometria, estatística e machine learning — em que cada uma difere 5
1.3. O ecossistema Python para econometria 5
1.4. O que você vai construir nesta apostila 5
Capítulo 2 — O Modelo de Regressão Linear Simples 6
2.1. Modelo populacional vs. modelo estimado 6
2.2. O método dos Mínimos Quadrados Ordinários (OLS) 7
2.3. Primeiro modelo: a função consumo keynesiana 8
2.4. Exemplo Resolvido — Interpretando os coeficientes da função consumo 8
Capítulo 3 — As Premissas de Gauss-Markov e o Teorema BLUE 10
3.1. Por que "só rodar a regressão" não basta 10
3.2. As seis premissas, uma a uma 10
3.3. A premissa que mais importa: exogeneidade 11
3.4. Exemplo Resolvido — Diagnosticando premissas a partir de um gráfico de resíduos 11
Capítulo 4 — Interpretando a Saída da Regressão 12
4.1. O `summary()` completo 12
4.2. R^2 — o quanto o modelo explica 12
4.3. Erro padrão, estatística t e p-valor 13
4.4. Intervalos de confiança 13
4.5. F-statistic — significância conjunta 13
4.6. Extraindo cada peça programaticamente 14
4.7. Exemplo Resolvido — Lendo uma saída de regressão fornecida como texto 14
Capítulo 5 — Testes de Hipótese em Regressão 15
5.1. A lógica de um teste de hipótese 15
5.2. Erro Tipo I e Tipo II 15
5.3. Teste t vs. teste F — quando usar cada um 16
5.4. O debate sobre o limiar de "0,05" 16
5.5. Exemplo Resolvido — Testando a Curva de Phillips 16
Capítulo 6 — Regressão Múltipla 18
6.1. Por que adicionar mais variáveis 18
6.2. R^2 ajustado como critério de comparação 18
6.3. Exemplo Resolvido — Comparando regressão simples e múltipla 18
Capítulo 7 — Correlação Não é Causalidade 19
7.1. Por que este é o capítulo mais importante da apostila 19
7.2. Viés de variável omitida (Omitted Variable Bias) 20
7.3. Causalidade reversa 20
7.4. O padrão-ouro: desenho experimental vs. dados observacionais 20
7.5. Exemplo Resolvido — Identificando um confounder plausível 21
Capítulo 8 — Multicolinearidade 22
8.1. O que é e por que é um problema 22
8.2. Sintomas e o VIF (Variance Inflation Factor) 22
8.3. O que fazer diante de multicolinearidade 23
8.4. Exemplo Resolvido — Diagnosticando e corrigindo multicolinearidade 23
Capítulo 9 — Heterocedasticidade 24
9.1. Revisitando o conceito 24

9.2. Consequência: coeficientes certos, confiança errada 24
9.3. Testando heterocedasticidade — Breusch-Pagan e White 25
9.4. Correção: erros-padrão robustos 25
9.5. Exemplo Resolvido — Heterocedasticidade no gasto com alimentação 25
Capítulo 10 — Variáveis Dummy e a Equação de Mincer 27
10.1. Codificando categorias como números 27
10.2. A armadilha da dummy (dummy trap) 27
10.3. A Equação de Mincer: retornos à educação 27
10.4. Interpretando o coeficiente de uma dummy em modelo log-linear 27
10.5. Interação entre dummy e variável contínua 28
10.6. Exemplo Resolvido — Decompondo o hiato salarial estimado 28
Capítulo 11 — Transformações Logarítmicas e Elasticidades 29
11.1. Por que usar logaritmo 29
11.2. As quatro combinações log/nível e suas interpretações 30
11.3. Exemplo clássico: elasticidade-renda da demanda (Curva de Engel) 30
11.4. Cuidado com valores zero ou negativos 30
11.5. Exemplo Resolvido — Elasticidade-preço da demanda 30
Capítulo 12 — Autocorrelação e a Curva de Phillips 32
12.1. O que é autocorrelação e por que é comum em séries temporais 32
12.2. Testando autocorrelação: Durbin-Watson 32
12.3. Correção: erros-padrão de Newey-West 32
12.4. Exemplo Resolvido — A Curva de Phillips completa, com diagnóstico 33
Capítulo 13 — Dados em Painel 34
13.1. O que é um painel e por que ele resolve parte do problema de exogeneidade 34
13.2. Pooled OLS vs. Efeitos Fixos 34
13.3. Efeitos fixos vs. efeitos aleatórios: o Teste de Hausman 35
13.4. Exemplo Resolvido — Comparando Pooled, FE e RE na prática 35
Capítulo 14 — Séries Temporais: Estacionariedade 37
14.1. Por que estacionariedade importa 37
14.2. Raiz unitária e passeio aleatório 37
14.3. O teste ADF (Augmented Dickey-Fuller) 37
14.4. O que fazer com uma série não estacionária: diferenciação 37
14.5. Exemplo Resolvido — Diagnosticando estacionariedade da Selic 37
Capítulo 15 — ARIMA, Cointegração, VAR e Causalidade de Granger 39
15.1. Modelos ARIMA 39
15.2. Cointegração: quando duas séries não estacionárias “andam juntas” 40
15.3. Modelos VAR e Função Impulso-Resposta 40
15.4. Causalidade de Granger 41
15.5. Exemplo Resolvido — Selic causa IPCA no sentido de Granger? 41
Capítulo 16 — Exercícios para Executar (na mão) 42
Exercício 1 — Interpretando um coeficiente log-linear 42
Exercício 2 — Calculando um resíduo manualmente 42
Exercício 3 — Lendo uma saída de regressão fornecida como texto 42
Exercício 4 — Identificando violação de premissa a partir de um gráfico descrito 42
Exercício 5 — Correlação vs. causalidade num caso concreto 42
Exercício 6 — Comparando dois intervalos de confiança 43
Capítulo 17 — Exercícios para Executar (em código) 43
Exercício 1 — Sua primeira regressão simples 43
Exercício 2 — Esquecendo o intercepto de propósito 43
Exercício 3 — Regressão por fórmula vs. por matriz 43
Exercício 4 — Intervalos de confiança em diferentes níveis 43
Exercício 5 — Regressão múltipla com três regressores 43
Exercício 6 — Calculando VIF 43

Exercício 7 — Teste de Breusch-Pagan 43
Exercício 8 — Erros-padrão robustos 43
Exercício 9 — Durbin-Watson em dados reais 43
Exercício 10 — Erros-padrão de Newey-West (HAC) 44
Exercício 11 — Variável dummy simples 44
Exercício 12 — Termo de interação 44
Exercício 13 — Log-linear e a correção de Halvorsen-Palmquist 44
Exercício 14 — Elasticidade em log-log 44
Exercício 15 — Teste ADF em duas séries diferentes 44
Exercício 16 — Teste de cointegração 44
Exercício 17 — Ajustando um ARIMA e prevendo 44
Exercício 18 — Selecionando a melhor ordem ARIMA por AIC 44
Exercício 19 — Modelo VAR e Função Impulso-Resposta 44
Exercício 20 — Causalidade de Granger nas duas direções 44
Exercício 21 — Painel: Pooled OLS vs. Efeitos Fixos 44
Exercício 22 — Teste de Hausman 45
Capítulo Final — Resumo 45
Capítulo 18 — Erros Comuns e Teste Rápido 45
18.1. Os erros mais comuns em econometria aplicada 45
18.2. Teste rápido: verdadeiro ou falso (com gabarito) 46
Próxima apostila 46

Como Usar Esta Apostila

A quarta apostila da série trata de **inferência estatística aplicada a dados econômicos**. Pressupõe as Apostilas 1 a 3 e noções de estatística (média, variância, distribuição normal). O desafio da econometria não é rodar a regressão — isso é uma linha de código — e sim saber *quando o resultado merece confiança*. Por isso, ao final de cada capítulo há um bloco de **aprofundamento** que usa **simulação de Monte Carlo**: criamos dados cujo mecanismo verdadeiro conhecemos, repetimos o experimento milhares de vezes e *vemos* o que acontece com o estimador. Assim, "viés", "erro-padrão", "erro tipo I" e "multicolinearidade" deixam de ser palavras e viram números que você pode reproduzir (todos os resultados impressos vêm da execução do código, com semente fixa).

Exemplo resolvido — modelo
Um experimento numérico, executado, com a interpretação.

Intuição
A ideia por trás do conceito, sem fórmula.

Aprofundando a teoria — modelo
A dedução ou o resultado formal.

Atenção — erro comum
Um erro que gera conclusões erradas com cara de rigor.

No mercado brasileiro
Como o conceito aparece em trabalhos aplicados e em análises de política econômica.

O fluxo de uma análise econométrica. (1) Formular a *pergunta* e o *mecanismo* (2) escolher o *modelo* e as variáveis; (3) coletar e limpar os dados; (4) *estimar* (5) *diagnosticar* as premissas; (6) corrigir (erros robustos, outras variáveis, transformações); (7) *interpretar* com cuidado (efeito, incerteza, causalidade). Os passos 5 e 7 são os que separam análise de ritual.

Capítulo 1 — O que é Econometria

1.1. Um pouco de história

O termo "econometria" foi cunhado pelo economista norueguês Ragnar Frisch em 1926, e a **Econometric Society** foi fundada em 1930 por Frisch, Irving Fisher e Charles Roos, com o objetivo declarado de unir teoria econômica, matemática e estatística num único corpo de método. Frisch e Jan Tinbergen dividiram o primeiro Prêmio Nobel de Economia, em 1969, precisamente por terem desenvolvido e aplicado os primeiros modelos econométricos dinâmicos.

A motivação original era simples de enunciar e difícil de resolver: a teoria econômica produz afirmações qualitativas ("um aumento da renda eleva o consumo") ou, no máximo, modelos matemáticos com parâmetros livres ("consumo é uma função linear da renda, com inclinação β"). Mas a teoria, sozinha, não diz **qual é o valor de β** no mundo real, nem se a relação proposta realmente se sustenta quando confrontada com dados observados. Preencher essa lacuna — estimar parâmetros, testar hipóteses, quantificar incerteza — é o objeto da econometria.

Vale registrar a definição clássica, ainda citada em praticamente todo curso de graduação, do próprio Frisch: econometria não é matemática aplicada à economia, nem é estatística aplicada a dados econômicos — é a **fusão das três coisas** (teoria econômica, matemática e estatística) em uma disciplina própria, com problemas e métodos que nenhuma das três, isoladamente, resolveria.

1.2. Econometria, estatística e machine learning — em que cada uma difere

É comum confundir as três, porque compartilham ferramentas (regressão, por exemplo, aparece nas três). A diferença está no **objetivo**:

Disciplina	Pergunta central	Prioridade	Exemplo típico
Estatística pura	Como descrever/inferir propriedades de uma população a partir de uma amostra?	Rigor matemático, generalidade	Testar se duas médias são diferentes
Econometria	Qual é a relação **causal ou estrutural** entre variáveis econômicas, e com que grau de confiança?	Interpretabilidade dos parâmetros (β tem significado econômico)	Estimar a propensão marginal a consumir
Machine learning	Como prever y da forma mais precisa possível, mesmo sem entender o mecanismo?	Poder preditivo fora da amostra	Prever inadimplência de um cliente

Um economista que estima consumo $= \beta_0 + \beta_1 \cdot$ renda não está interessado apenas em prever o consumo do mês que vem — está interessado em **interpretar** β_1 como a propensão marginal a consumir, um parâmetro com significado teórico (a Teoria Geral de Keynes prevê que $0 < \beta_1 < 1$). Um cientista de dados que treina uma floresta aleatória para prever inadimplência normalmente não se importa em interpretar cada "coeficiente" — só que o modelo acerte. Essa diferença de propósito é o que faz um economista escolher `statsmodels` (que expõe erros-padrão, p-valores, testes de hipótese) em vez de `scikit-learn` (otimizado para previsão, com interpretabilidade secundária) como ferramenta primária.

1.3. O ecossistema Python para econometria

Biblioteca	Ponto forte	Quando usar
statsmodels	Saída estatística completa (erros-padrão, testes, diagnósticos), sintaxe próxima da literatura econométrica	Regressão OLS/GLS, séries temporais (ARIMA, VAR), testes de hipótese
linearmodels	Estende o statsmodels para dados em painel (efeitos fixos/aleatórios) e variáveis instrumentais	Painéis de países/empresas, IV/2SLS
scikit-learn	Algoritmos de machine learning, validação cruzada, pipelines	Previsão pura, quando interpretar β não é o objetivo
scipy.stats	Testes estatísticos genéricos (normalidade, distribuições)	Complementa o statsmodels em diagnósticos
numpy/pandas	Álgebra linear e manipulação tabular	Base de tudo o que vem acima

Nesta apostila trabalharemos majoritariamente com `statsmodels`, porque sua filosofia — expor toda a incerteza estatística por trás de uma estimativa, não apenas o valor pontual — é exatamente o que a formação econômica exige.

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.tsa.stattools import adfuller
from bcb import sgs
```

1.4. O que você vai construir nesta apostila

Ao final, você será capaz de: estimar e interpretar regressões lineares simples e múltiplas; reconhecer e testar as premissas por trás da regressão (Gauss-Markov); diagnosticar e corrigir heterocedasticidade, multicolinearidade

e autocorrelação; interpretar variáveis dummy e transformações logarítmicas; e aplicar os principais modelos de séries temporais (estacionariedade, ARIMA, cointegração, VAR, causalidade de Granger) e de dados em painel usados em pesquisa aplicada e em bancos centrais.

O fio condutor será sempre um exemplo econômico concreto — a função consumo keynesiana, a Curva de Phillips, a equação de Mincer para retornos à educação — porque econometria sem uma pergunta econômica por trás é apenas álgebra linear.

> **Aprofundando a teoria — OLS em forma matricial: o que o `summary()` faz por baixo**
> Para o modelo $y = X\beta + \varepsilon$, com X a matriz de regressores (incluindo uma coluna de 1 para o intercepto), o estimador de Mínimos Quadrados Ordinários minimiza a soma dos quadrados dos resíduos, $\min_\beta (y - X\beta)^T(y - X\beta)$. A condição de primeira ordem dá as **equações normais** $X^T X\hat{\beta} = X^T y$, e portanto
>
> $$\hat{\beta} = (X^T X)^{-1} X^T y \quad \text{e} \quad \widehat{\mathrm{Var}}(\hat{\beta}) = \hat{\sigma}^2 (X^T X)^{-1}, \quad \hat{\sigma}^2 = \frac{\hat{\varepsilon}^T \hat{\varepsilon}}{n - k}$$
>
> No caso simples ($y = \beta_0 + \beta_1 x$), isso se reduz a $\hat{\beta}_1 = \frac{\mathrm{Cov}(x,y)}{\mathrm{Var}(x)}$ e $\hat{\beta}_0 = |y| - \hat{\beta}_1 |x|$. O `statsmodels` faz exatamente essas contas; o código abaixo refaz tudo "à mão" com NumPy para demonstrar que não há mistério.

```
import statsmodels.api as sm
rng_e = np.random.default_rng(10)
n_e = 200
renda_e = rng_e.uniform(2, 20, n_e)                                  # renda em milhares de
R$
consumo_e = 1.5 + 0.8 * renda_e + rng_e.normal(0, 1.2, n_e)         # mecanismo verdadeiro:
b0 = 1.5, b1 = 0.8

X_e = np.column_stack([np.ones(n_e), renda_e])
beta_e = np.linalg.solve(X_e.T @ X_e, X_e.T @ consumo_e)             # equações normais
resid_e = consumo_e - X_e @ beta_e
sigma2_e = resid_e @ resid_e / (n_e - 2)
ep_e = np.sqrt(np.diag(sigma2_e * np.linalg.inv(X_e.T @ X_e)))

ajuste_e = sm.OLS(consumo_e, X_e).fit()
print("À mão       : b0 = %.4f, b1 = %.4f | erros-padrão: %.4f, %.4f" % (*beta_e, *ep_e))
print("statsmodels : b0 = %.4f, b1 = %.4f | erros-padrão: %.4f, %.4f" % (*ajuste_e.params,
*ajuste_e.bse))
print("Cov(x,y)/Var(x) =", round(np.cov(renda_e, consumo_e, ddof=1)[0, 1] / np.var(renda_e,
ddof=1), 4))
```

```
À mão       : b0 = 1.4624, b1 = 0.8015 | erros-padrão: 0.1857, 0.0155
statsmodels : b0 = 1.4624, b1 = 0.8015 | erros-padrão: 0.1857, 0.0155
Cov(x,y)/Var(x) = 0.8015
```

Capítulo 2 — O Modelo de Regressão Linear Simples

2.1. Modelo populacional vs. modelo estimado

A distinção mais importante deste capítulo — e a que mais confunde iniciantes — é entre o **modelo populacional** (o que é verdadeiro no mundo, mas nunca observamos diretamente) e o **modelo estimado** (o que calculamos a partir de uma amostra de dados).

O modelo populacional de regressão linear simples é:

$$y_i = \beta_0 + \beta_1 x_i + u_i$$

onde y_i é a variável dependente (o que queremos explicar), x_i é a variável explicativa, β_0 e β_1 são os **parâmetros populacionais verdadeiros** (desconhecidos, fixos, não observados), e u_i é o **termo de erro** — tudo o que afeta y_i e não é capturado por x_i (variáveis omitidas, erro de medida, aleatoriedade genuína).

Como nunca observamos β_0 e β_1 diretamente, o que fazemos é **estimá-los** a partir de uma amostra de n observações, produzindo:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

O "chapéu" ($\hat{}$) é a convenção universal em econometria para indicar "isto é uma estimativa, não o valor verdadeiro". A diferença entre o valor observado e o valor predito é o **resíduo**:

$$\hat{u}_i = y_i - \hat{y}_i$$

O resíduo $\hat{u}_i$ é uma **estimativa observável** do erro u_i (não observável) — outra confusão comum é tratá-los como sinônimos.

2.2. O método dos Mínimos Quadrados Ordinários (OLS)

A pergunta natural é: como escolher $\hat{\beta}_0$ e $\hat{\beta}_1$? O critério de **Mínimos Quadrados Ordinários** (*Ordinary Least Squares*, OLS) escolhe os valores que **minimizam a soma dos quadrados dos resíduos**:

$$\min_{\hat{\beta}_0, \hat{\beta}_1} \sum_{i=1}^{n} \hat{u}_i^2 = \sum_{i=1}^{n} (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

Por que **quadrados**, e não a soma dos resíduos em valor absoluto, por exemplo? Duas razões práticas: (1) elevar ao quadrado penaliza desproporcionalmente erros grandes, o que geralmente é desejável; e (2) a função objetivo resultante é diferenciável em toda parte, o que permite resolver o problema analiticamente (derivando e igualando a zero), em vez de recorrer a métodos numéricos iterativos. Resolvendo esse sistema (as chamadas **equações normais**), chega-se a uma fórmula fechada.

A derivação, passo a passo. Chame a soma dos quadrados dos resíduos de $S\left(\hat{\beta}_0, \hat{\beta}_1\right)$. Minimizar uma função de duas variáveis exige que **ambas** as derivadas parciais sejam nulas simultaneamente (condição de primeira ordem):

$$\frac{\partial S}{\partial \hat{\beta}_0} = -2 \sum_{i=1}^{n} \left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i\right) = 0$$

$$\frac{\partial S}{\partial \hat{\beta}_1} = -2 \sum_{i=1}^{n} x_i \left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i\right) = 0$$

A primeira equação, dividida por $-2n$, diz que a soma dos resíduos precisa ser exatamente zero — é **por construção do próprio método**, não uma hipótese adicional, que a reta de OLS sempre passa pelo ponto médio $(\bar{x}, \bar{y})$:

$$\bar{y} - \hat{\beta}_0 - \hat{\beta}_1 \bar{x} = 0 \quad \Rightarrow \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Substituindo esse $\hat{\beta}_0$ na segunda equação e isolando $\hat{\beta}_1$ (álgebra que aqui se omite, mas que é apenas expandir o produto e agrupar termos em $\hat{\beta}_1$), chega-se à fórmula fechada abaixo. O ponto que costuma passar despercebido: essas duas equações não são um "truque de cálculo" isolado — são o motivo estrutural pelo qual os **resíduos de uma regressão OLS sempre somam zero** e **nunca são correlacionados com o regressor x** (a segunda equação, reescrita, diz exatamente $\sum x_i \hat{u}_i = 0$). Essas duas propriedades reaparecerão no Capítulo 3 como consequência direta — não coincidência — do próprio critério de minimização.

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2} = \frac{\mathrm{Cov}(x, y)}{\mathrm{Var}(x)}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Repare que $\hat{\beta}_1$ é, essencialmente, a covariância entre x e y escalada pela variância de x — o que já antecipa por que regressão simples e correlação estão tão intimamente ligadas (voltaremos a isso, e a por que são conceitos diferentes, no Capítulo 7).

2.3. Primeiro modelo: a função consumo keynesiana

A relação entre consumo e renda é um dos pontos de partida clássicos da macroeconomia — a "função consumo" de Keynes, formalizada como $C = C_0 + c \cdot Y$, onde C_0 é o consumo autônomo (o que se consome mesmo com renda zero, financiado por poupança ou crédito) e c é a **propensão marginal a consumir** (PMgC), a fração de cada unidade adicional de renda que é destinada ao consumo em vez de poupança.

```
# Dados simulados de consumo e renda (função consumo keynesiana)
np.random.seed(42)
renda = np.arange(1000, 11000, 1000)
consumo = 200 + 0.8 * renda + np.random.normal(0, 100, len(renda))

df = pd.DataFrame({"renda": renda, "consumo": consumo})

# statsmodels NÃO adiciona o intercepto (beta_0) automaticamente —
# é preciso pedir explicitamente com sm.add_constant()
X = sm.add_constant(df["renda"])
y = df["consumo"]

modelo = sm.OLS(y, X).fit()
print(modelo.summary())
```

Atenção — a armadilha mais comum de quem começa com statsmodels: esquecer `sm.add_constant(X)`. Sem ela, o `sm.OLS` estima um modelo **forçado a passar pela origem** ($\hat{\beta}_0 = 0$), o que quase nunca é o que se pretende economicamente (implicaria consumo zero para renda zero) e distorce todos os coeficientes, inclusive o R^2. Isso é diferente de `smf.ols("consumo ~ renda", data=df)` (a interface por fórmula, no estilo R), que adiciona o intercepto automaticamente — as duas interfaces do statsmodels têm essa diferença de comportamento padrão, e vale saber qual delas você está usando.

```
# Interface por fórmula — adiciona o intercepto por padrão
modelo2 = smf.ols("consumo ~ renda", data=df).fit()
print(modelo2.params)  # idênticos aos de "modelo" acima
```

2.4. Exemplo Resolvido — Interpretando os coeficientes da função consumo

Enunciado: rodando o modelo do item 2.3, o statsmodels retorna aproximadamente $\hat{\beta}_0 = 226{,}72$ e $\hat{\beta}_1 = 0{,}796$. Interprete cada um em português econômico, e calcule o consumo previsto para uma renda de `R\$ 7.500`.

Solução comentada:

```
beta0, beta1 = modelo.params["const"], modelo.params["renda"]
print(f"Consumo autônomo (const): {beta0:.2f}")
print(f"Propensão marginal a consumir (renda): {beta1:.4f}")

renda_nova = 7500
consumo_previsto = beta0 + beta1 * renda_nova
print(f"Consumo previsto para renda = {renda_nova}: {consumo_previsto:.2f}")
```

- $\hat{\beta}_0 = 226{,}72$: é o consumo previsto quando a renda é zero — o **consumo autônomo**. Note que essa interpretação literal só faz sentido dentro (ou perto) do intervalo de renda observado nos dados (`R\$ 1.000` a `R\$ 10.000`); extrapolar para renda = 0 é um exercício algébrico, não necessariamente uma afirmação econômica confiável (esse problema tem nome — **extrapolação fora da amostra** — e reaparecerá na Apostila 6/7 ao falar de previsão).

- $\hat{\beta}_1 = 0{,}796$: para cada `R$ 1,00` adicional de renda, o consumo aumenta, em média, `R$ 0,796`, **mantendo tudo o mais constante** (aqui, "tudo o mais" é vazio, porque é regressão simples — a expressão ganha peso reforçado no Capítulo 6, regressão múltipla). O valor está dentro do intervalo teórico esperado pela Teoria Geral de Keynes ($0 < c < 1$), o que é uma checagem de sanidade útil: um $\hat{\beta}_1$ estimado fora desse intervalo (negativo, ou maior que 1) pediria investigação antes de aceitar o resultado.
- O consumo previsto para renda `R$ 7.500` é $226{,}72 + 0{,}796 \times 7500 \approx \text{R\$ } 6.196{,}72$.

Aprofundando a teoria — o que significa "não viesado" e "erro-padrão": a distribuição amostral
O estimador $\hat{\beta}_1$ é uma **variável aleatória**: se coletássemos outra amostra, obteríamos outro valor. Dizer que o OLS é **não viesado** é dizer que, *na média sobre infinitas amostras*, $E\left[\hat{\beta}_1\right] = \beta_1$. O **erro-padrão** é o desvio padrão desse estimador entre amostras. Como só temos uma amostra, o erro-padrão é *estimado* pela fórmula; com simulação, podemos conferir se a fórmula acerta. O experimento: 3.000 amostras de 60 observações do mesmo mecanismo.

```
rng_mc = np.random.default_rng(1)
b1_verdadeiro, n_mc, reps = 0.8, 60, 3000
b1_hat, ep_hat = np.empty(reps), np.empty(reps)
for r in range(reps):
    x = rng_mc.uniform(2, 20, n_mc)
    y = 1.5 + b1_verdadeiro * x + rng_mc.normal(0, 1.2, n_mc)
    Xm = np.column_stack([np.ones(n_mc), x])
    bh = np.linalg.solve(Xm.T @ Xm, Xm.T @ y)
    res = y - Xm @ bh
    b1_hat[r] = bh[1]
    ep_hat[r] = np.sqrt((res @ res / (n_mc - 2)) * np.linalg.inv(Xm.T @ Xm)[1, 1])
print(f"Verdadeiro: {b1_verdadeiro} | média das {reps} estimativas: {b1_hat.mean():.4f}
(viés ≈ {b1_hat.mean() - b1_verdadeiro:+.4f})")
print(f"Desvio padrão REAL das estimativas: {b1_hat.std(ddof=1):.4f} | média do erro-padrão
ESTIMADO: {ep_hat.mean():.4f}")
```

```
Verdadeiro: 0.8 | média das 3000 estimativas: 0.7995 (viés ≈ -0.0005)
Desvio padrão REAL das estimativas: 0.0305 | média do erro-padrão ESTIMADO: 0.0301
```

Intuição
O OLS acerta **em média** (viés quase zero) e o erro-padrão calculado pela fórmula reproduz a dispersão *real* das estimativas: é por isso que se pode confiar na inferência (intervalos e testes) **quando as premissas valem**. Os próximos experimentos mostram o que acontece quando elas falham.

Exemplo resolvido — quando a premissa de linearidade falha: RESET e normalidade dos resíduos
O verdadeiro efeito é *quadrático* (retornos decrescentes), mas ajustamos uma reta. Dois diagnósticos formais: o teste **RESET de Ramsey** (a forma funcional está correta?) e o de **Jarque-Bera** (os resíduos são normais?).

```
from statsmodels.stats.diagnostic import linear_reset
from statsmodels.stats.stattools import jarque_bera
rng_q = np.random.default_rng(5)
x_q = rng_q.uniform(0, 10, 300)
y_q = 2 + 3 * x_q - 0.25 * x_q ** 2 + rng_q.normal(0, 1, 300)
reta = sm.OLS(y_q, sm.add_constant(x_q)).fit()
curva = sm.OLS(y_q, sm.add_constant(np.column_stack([x_q, x_q ** 2]))).fit()
print(f"R² da reta: {reta.rsquared:.3f} | R² do modelo quadrático: {curva.rsquared:.3f}")
print(f"RESET (reta): p-valor = {linear_reset(reta, power=2, use_f=True).pvalue:.2e} ->
{'rejeita' if linear_reset(reta, power=2, use_f=True).pvalue < 0.05 else 'não rejeita'} a
forma linear")
print(f"RESET (quadrático): p-valor = {linear_reset(curva, power=2,
```

```
use_f=True).pvalue:.3f}")
print(f"Jarque-Bera dos resíduos da reta: p = {jarque_bera(reta.resid)[1]:.3f} | do modelo
quadrático: p = {jarque_bera(curva.resid)[1]:.3f}")
```

```
R² da reta: 0.320 | R² do modelo quadrático: 0.851
RESET (reta): p-valor = 4.33e-100 -> rejeita a forma linear
RESET (quadrático): p-valor = 0.290
Jarque-Bera dos resíduos da reta: p = 0.000 | do modelo quadrático: p = 0.094
```

Capítulo 3 — As Premissas de Gauss-Markov e o Teorema BLUE

3.1. Por que "só rodar a regressão" não basta

O estimador de OLS existe e produz um número mesmo quando os dados são inadequados — o computador não vai recusar rodar uma regressão só porque as premissas estatísticas por trás dela foram violadas. É por isso que a etapa de **verificar premissas** (Capítulos 3, 8 e 9 desta apostila) não é burocracia acadêmica: é o que separa uma estimativa confiável de um número sem sentido estatístico, ainda que o `summary()` pareça, à primeira vista, igualmente "sério" nos dois casos.

O **Teorema de Gauss-Markov** afirma que, sob um conjunto de seis premissas, o estimador de OLS é o **BLUE** — *Best Linear Unbiased Estimator* (Melhor Estimador Linear Não Viesado). Cada palavra desse acrônimo carrega um significado técnico preciso:

- **Linear**: $\hat{\beta}$ é uma função linear dos valores de y.
- **Unbiased (não viesado)**: em média, ao longo de infinitas amostras hipotéticas, $\hat{\beta}$ acerta o valor verdadeiro β: $E[\hat{\beta}] = \beta$.
- **Best (melhor)**: entre todos os estimadores lineares não viesados, o OLS é o que tem a **menor variância** — ou seja, a estimativa mais precisa possível dentro dessa classe.

3.2. As seis premissas, uma a uma

#	Premissa	Enunciado técnico	Violação comum em economia
1	**Linearidade nos parâmetros**	$y = \beta_0 + \beta_1 x + u$ (linear nos β's, não necessariamente em x)	Relação verdadeira é não-linear (ex: retornos decrescentes de capital) — resolve-se com transformações (Capítulo 11)
2	**Amostragem aleatória**	As observações (x_i, y_i) são extraídas aleatoriamente da população	Amostra viesada (ex: só empresas que sobreviveram — *survivorship bias*)
3	**Ausência de colinearidade perfeita**	Nenhuma variável explicativa é combinação linear exata de outra	Duas variáveis medindo essencialmente a mesma coisa (Capítulo 8)
4	**Exogeneidade / média condicional zero**	$E[u \mid x] = 0$ — o erro não está correlacionado com x	Variável omitida correlacionada com x e com y (Capítulo 7) — a mais grave e mais difícil de garantir
5	**Homocedasticidade**	$\mathrm{Var}(u \mid x) = \sigma^2$ constante para todo x	Variância do erro cresce com o nível de x (ex: erro de previsão de gasto cresce com a renda) (Capítulo 9)

#	Premissa	Enunciado técnico	Violação comum em economia
6	**Normalidade dos erros** *(opcional, para inferência em amostras pequenas)*	$u \sim N(0, \sigma^2)$	Erros com caudas pesadas ou assimetria — menos grave em amostras grandes (Teorema Central do Limite)

As premissas 1 a 5 garantem que o OLS é BLUE (Gauss-Markov não exige normalidade). A premissa 6 é necessária apenas para que os testes de hipótese (t, F) tenham validade **exata** em amostras pequenas; em amostras grandes, o Teorema Central do Limite garante uma aproximação razoável mesmo sem normalidade exata dos erros.

3.3. A premissa que mais importa: exogeneidade

Das seis, a premissa 4 (exogeneidade, $E[u|x] = 0$) é sistematicamente a mais violada em dados observacionais econômicos, e a mais consequente: sua violação gera **viés**, que não desaparece nem com uma amostra infinita — diferente, por exemplo, da falta de eficiência causada por heterocedasticidade, que "só" torna os erros-padrão incorretos. Voltaremos a isso com profundidade no Capítulo 7 (correlação vs. causalidade), porque é o ponto onde mais pesquisa aplicada em economia tropeça.

3.4. Exemplo Resolvido — Diagnosticando premissas a partir de um gráfico de resíduos

Enunciado: você roda uma regressão de gasto das famílias com alimentação (`gasto_alimentacao`) contra renda (`renda`), e plota os resíduos ($\hat{u}_i$, eixo y) contra os valores previstos ($\hat{y}_i$, eixo x). O gráfico mostra os pontos formando um "leque" — dispersos bem próximos de zero para valores previstos baixos, e cada vez mais espalhados (para cima e para baixo) conforme o valor previsto cresce. Qual premissa de Gauss-Markov está sendo violada, e qual é a consequência prática?

Solução comentada:

O padrão de "leque" (variância dos resíduos crescendo com o nível da variável prevista) é a assinatura visual clássica de **heterocedasticidade** — violação da premissa 5. Faz sentido economicamente: famílias de renda baixa têm pouca margem de escolha no gasto com alimentação (o piso de subsistência limita a variação), enquanto famílias de renda alta têm muito mais liberdade — algumas gastam pouco (poupam ou têm hábitos frugais), outras gastam muito (alimentação sofisticada) — logo a dispersão do gasto em torno da média prevista aumenta com a renda.

A consequência prática, importante para não confundir: heterocedasticidade **não torna $\hat{\beta}_1$ viesado** (o Gauss-Markov ainda garante um estimador não viesado sem a premissa 5) — mas torna os **erros-padrão calculados pelo OLS incorretos**, o que invalida os testes t e F e os intervalos de confiança relatados no `summary()`. Ou seja: o coeficiente pode estar certo, mas a "confiança" que você tem nele (o p-valor) pode estar errada. A correção — erros-padrão robustos — é o assunto do Capítulo 9.

> **Aprofundando a teoria — R^2, F, t e intervalo de confiança: refazendo o `summary()` na mão**
> A variação total se decompõe em explicada e residual: $\text{SST} = \text{SSE} + \text{SSR}$ (soma dos quadrados total = explicada + dos resíduos). Daí $R^2 = \frac{\text{SSE}}{\text{SST}} = 1 - \frac{\text{SSR}}{\text{SST}}$ e $R^2_{\text{aj}} = 1 - (1 - R^2)\frac{n-1}{n-k}$, que penaliza regressores inúteis. A estatística $t = \frac{\hat{\beta}_j}{\text{ep}(\hat{\beta}_j)}$ testa $H_0 : \beta_j = 0$; o **intervalo de confiança** de 95% é $\hat{\beta}_j \pm t_{0{,}975;n-k} \cdot \text{ep}(\hat{\beta}_j)$; a estatística F testa a significância *conjunta*: $F = \frac{\text{SSE}/(k-1)}{\text{SSR}/(n-k)}$. Tudo calculável com poucas linhas:

```
from scipy import stats as sps
sst = ((consumo_e - consumo_e.mean()) ** 2).sum()
ssr = (resid_e ** 2).sum()
sse = sst - ssr
k_e = 2
r2 = sse / sst
r2_aj = 1 - (1 - r2) * (n_e - 1) / (n_e - k_e)
t_b1 = beta_e[1] / ep_e[1]
```

```
p_b1 = 2 * (1 - sps.t.cdf(abs(t_b1), n_e - k_e))
ic = (beta_e[1] - sps.t.ppf(0.975, n_e - k_e) * ep_e[1], beta_e[1] + sps.t.ppf(0.975, n_e -
k_e) * ep_e[1])
f_stat = (sse / (k_e - 1)) / (ssr / (n_e - k_e))
print(f"SST = {sst:.1f} = SSE {sse:.1f} + SSR {ssr:.1f} | R² = {r2:.4f} (statsmodels:
{ajuste_e.rsquared:.4f}) | R² ajustado = {r2_aj:.4f}")
print(f"t(b1) = {t_b1:.2f} (statsmodels: {ajuste_e.tvalues[1]:.2f}) | p = {p_b1:.2e} | IC
95%: ({ic[0]:.4f}, {ic[1]:.4f})")
print(f"F = {f_stat:.1f} (statsmodels: {ajuste_e.fvalue:.1f}) | note: com um só regressor,
F = t² = {t_b1 ** 2:.1f}")
```

```
SST = 3527.8 = SSE 3283.5 + SSR 244.4 | R² = 0.9307 (statsmodels: 0.9307) | R² ajustado =
0.9304
t(b1) = 51.58 (statsmodels: 51.58) | p = 0.00e+00 | IC 95%: (0.7709, 0.8322)
F = 2660.5 (statsmodels: 2660.5) | note: com um só regressor, F = t² = 2660.5
```

Atenção — erro comum
R² alto não é sinônimo de bom modelo. Regredir duas séries que crescem no tempo produz R² próximo de 1 sem qualquer relação causal (capítulo 14). E um R² baixo não invalida o resultado: em dados micro (renda individual), R² de "0,10" a "0,30" é comum e os coeficientes podem ser estimados com precisão. O R² mede *ajuste* a causalidade e a adequação do modelo se julgam por outros meios.

Capítulo 4 — Interpretando a Saída da Regressão

4.1. O `summary()` completo

```
modelo = sm.OLS(y, X).fit()
print(modelo.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                 consumo   R-squared:                       0.996
Model:                             OLS   Adj. R-squared:                  0.995
Method:                  Least Squares   F-statistic:                     1782.
Date:                                    Prob (F-statistic):           2.58e-10
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        226.7169     67.233      3.372      0.010      69.812     383.622
renda          0.7959      0.019     42.214      0.000       0.752       0.840
==============================================================================
```

4.2. R² — o quanto o modelo explica

O **coeficiente de determinação** R^2 mede a proporção da variação total de y que é explicada pelo modelo:

$$R^2 = \frac{\text{SQE}}{\text{SQT}} = 1 - \frac{\text{SQR}}{\text{SQT}}$$

onde $\text{SQT} = \sum (y_i - \bar{y})^2$ é a soma dos quadrados totais (a variação de y em torno de sua média), $\text{SQE} = \sum (\hat{y}_i - \bar{y})^2$ é a soma dos quadrados explicados pelo modelo, e $\text{SQR} = \sum \hat{u}_i^2$ é a soma dos quadrados dos resíduos (o que sobra, não explicado). $R^2 = 0,996$ significa que "99,6"% da variação do consumo entre as observações é explicada pela variação da renda — um valor artificialmente alto aqui porque os dados foram simulados com pouco ruído; em dados reais de corte transversal com uma única variável explicativa, R^2 acima de "0,5" já costuma ser considerado alto.

Gotcha: R^2 **sempre aumenta** (ou, no limite, permanece igual) quando se adiciona qualquer variável explicativa nova ao modelo — mesmo uma variável sem qualquer relação teórica com y (ex: o número de letras do nome do presidente do Banco Central). Isso torna o R^2 simples um péssimo critério para decidir se vale a pena incluir uma variável adicional — para isso existe o R^2 ajustado:

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-k-1}$$

onde n é o número de observações e k o número de variáveis explicativas (sem contar a constante). O R^2 ajustado penaliza a inclusão de variáveis que não contribuem o suficiente, podendo até **cair** quando uma variável irrelevante é adicionada — o que o torna a métrica preferida para comparar modelos com número diferente de regressores.

4.3. Erro padrão, estatística t e p-valor

Cada coeficiente estimado $\hat{\beta}_1$ tem um **erro padrão** (std err), que mede a variabilidade esperada de $\hat{\beta}_1$ se repetíssemos o processo de amostragem inúmeras vezes. A partir dele, constrói-se a **estatística t** para testar a hipótese nula de que o coeficiente populacional verdadeiro é zero (isto é, que a variável não tem efeito):

$$t = \frac{\hat{\beta}_1 - 0}{\text{se}(\hat{\beta}_1)}$$

O **p-valor** é a probabilidade de observar uma estatística t tão extrema quanto a calculada (ou mais), **assumindo que a hipótese nula é verdadeira**. A convenção universal (mas arbitrária — ver Capítulo 5.4) é rejeitar H_0 quando $p < 0,05$: para `renda`, $p < 0,001$, uma evidência estatística muito forte de que a renda afeta o consumo.

Gotcha de interpretação, frequente até em trabalhos publicados: p-valor **não é** "a probabilidade de H_0 ser verdadeira". É a probabilidade de observar um dado tão extremo quanto o observado **dado que** H_0 é verdadeira — uma afirmação condicional, não a afirmação invertida. Confundir as duas é um dos erros conceituais mais comuns em estatística aplicada.

4.4. Intervalos de confiança

A coluna `[0.025  0.975]` do `summary()` mostra o **intervalo de confiança de 95%** para cada coeficiente:

$$\hat{\beta}_1 \pm t_{\alpha/2,\, n-k-1} \cdot \text{se}(\hat{\beta}_1)$$

Interpretação correta (e sutil): "se repetíssemos esse procedimento de amostragem muitas vezes, 95% dos intervalos construídos dessa forma conteriam o verdadeiro β_1". **Não** significa "há 95% de chance de o β_1 verdadeiro estar neste intervalo específico" — o β_1 verdadeiro é um número fixo (embora desconhecido), não uma variável aleatória; quem é aleatório é o intervalo, que muda a cada amostra.

```
modelo.conf_int()          # intervalos de confiança (95% por padrão)
modelo.conf_int(alpha=0.01)  # intervalo de 99%
```

4.5. F-statistic — significância conjunta

Enquanto o teste t avalia **um** coeficiente de cada vez, a estatística F testa a hipótese conjunta de que **todos** os coeficientes (exceto a constante) são simultaneamente zero:

$$H_0 : \beta_1 = \beta_2 = \ldots = \beta_k = 0 \quad \text{vs.} \quad H_1 : \text{pelo menos um} \;\; \beta_j \neq 0$$

`Prob (F-statistic)` extremamente baixo (`2.58e-10`) rejeita fortemente H_0 — o modelo, como um todo, tem poder explicativo estatisticamente significativo. Isso importa especialmente em regressão múltipla: é possível (embora incomum) que nenhum coeficiente individual seja significativo no teste t, mas o F conjunto seja significativo — sintoma típico de multicolinearidade (Capítulo 8).

4.6. Extraindo cada peça programaticamente

```
modelo.params         # coeficientes (Series indexada pelo nome da variável)
modelo.pvalues        # p-valores
modelo.rsquared       # R²
modelo.rsquared_adj   # R² ajustado
modelo.resid          # resíduos
modelo.fittedvalues   # valores preditos (y-chapéu)
modelo.conf_int()     # intervalos de confiança (95%)
modelo.bse            # erros padrão (bse = "beta standard errors")
modelo.fvalue         # estatística F
modelo.f_pvalue       # p-valor da estatística F
```

4.7. Exemplo Resolvido — Lendo uma saída de regressão fornecida como texto

Enunciado: um colega enviou apenas o texto abaixo, resultado de uma regressão de salário (em milhares de `R\$` por ano) contra anos de escolaridade e experiência, com $n = 200$ observações. Sem rodar nenhum código, responda: (a) qual o efeito estimado de mais um ano de escolaridade sobre o salário? (b) o coeficiente de experiência é estatisticamente significativo a 5%? (c) o modelo como um todo é significativo? (d) o que representa o `R-squared` de "0,312" neste contexto?

```
                coef    std err          t      P>|t|
------------------------------------------------------------------------------
const          8.2000      1.500      5.467      0.000
escolaridade    2.1000      0.310      6.774      0.000
experiencia     0.4500      0.290      1.552      0.122
==============================================================================
R-squared: 0.312           F-statistic: 44.68        Prob (F-statistic): 1.2e-16
```

Solução comentada:

(a) Mantendo a experiência constante, cada ano adicional de escolaridade está associado, em média, a um aumento de `R\$ "2.100,00"` por ano no salário (o coeficiente `2.100` está em milhares de reais).

(b) Não. O p-valor de `experiencia` é `"0,122"`, maior que `"0,05"` — não há evidência estatística suficiente, a 5% de significância, para rejeitar a hipótese de que o efeito da experiência sobre o salário é zero, **neste modelo e nesta amostra**. Importante: isso **não prova** que experiência não afeta salário na população — apenas que esta amostra não forneceu evidência forte o bastante para essa variável específica (possivelmente por colinearidade com escolaridade, ou por tamanho de amostra insuficiente para detectar um efeito verdadeiro, mas pequeno).

(c) Sim. `Prob (F-statistic) = 1.2e-16` é essencialmente zero — rejeita-se fortemente a hipótese conjunta de que escolaridade e experiência juntas não têm efeito nenhum sobre o salário, mesmo que, isoladamente, `experiencia` não tenha passado no teste t individual.

(d) $R^2 = 0,312$ significa que "31,2"% da variação do salário entre os 200 indivíduos da amostra é explicada pelas duas variáveis do modelo. Isso é um valor tipicamente razoável (não alto) para dados de corte transversal de indivíduos — salário depende de dezenas de fatores não observados (habilidade inata, rede de contatos, sorte, setor de atuação), então é esperado que a maior parte da variação ("68,8"%) fique no termo de erro.

> **Aprofundando a teoria — erro tipo I, poder e a distribuição do p-valor sob a hipótese nula**
> Se H_0 é *verdadeira*, o p-valor tem distribuição **uniforme** em $[0, 1]$: em 5% dos casos ele cai abaixo de "0,05" por puro acaso. É o **erro tipo I** — rejeitar uma hipótese nula verdadeira — e sua taxa é, por construção, o nível de significância α. O **poder** do teste é a probabilidade de rejeitar H_0 quando ela é *falsa* cresce com o tamanho do efeito, o tamanho da amostra e diminui com o ruído. Simulação: um regressor **sem nenhum efeito** (erro tipo I) e outros com efeitos crescentes (poder), em amostras de 100:

```
rng_p = np.random.default_rng(2)
reps_p, n_p = 4000, 100
for efeito in (0.0, 0.1, 0.2, 0.4):
    rejeita = 0
    for _ in range(reps_p):
        x = rng_p.normal(size=n_p)
        y = efeito * x + rng_p.normal(size=n_p)
        xc = x - x.mean()
        b = (xc @ (y - y.mean())) / (xc @ xc)
        res = y - y.mean() - b * xc
        ep = np.sqrt((res @ res / (n_p - 2)) / (xc @ xc))
        rejeita += abs(b / ep) > sps.t.ppf(0.975, n_p - 2)
    rot = "erro tipo I (efeito nulo)" if efeito == 0 else "poder"
    print(f"efeito verdadeiro = {efeito:.1f} -> rejeita H0 em {rejeita / reps_p:.1%} das
amostras ({rot})")
```

```
efeito verdadeiro = 0.0 -> rejeita H0 em 5.3% das amostras (erro tipo I (efeito nulo))
efeito verdadeiro = 0.1 -> rejeita H0 em 17.4% das amostras (poder)
efeito verdadeiro = 0.2 -> rejeita H0 em 50.1% das amostras (poder)
efeito verdadeiro = 0.4 -> rejeita H0 em 97.0% das amostras (poder)
```

Exemplo resolvido — o problema das comparações múltiplas
Se você testa 20 variáveis **sem nenhum efeito real** a 5% de significância, a probabilidade de encontrar *pelo menos uma* "significativa" é $1 - 0,95^{20} \approx 64\%$. É assim que "mineração de dados" produz "descobertas" espúrias. Correções simples: o ajuste de **Bonferroni** (usar α/m) ou, melhor, **pré-registrar** as hipóteses antes de olhar os dados.

```
print(f"P(ao menos 1 falso positivo em 20 testes a 5%) = {1 - 0.95 ** 20:.1%}")
print(f"Com Bonferroni (alfa = 0,05/20 = {0.05 / 20:.4f}): {1 - (1 - 0.05 / 20) **
20:.1%}")
```

```
P(ao menos 1 falso positivo em 20 testes a 5%) = 64.2%
Com Bonferroni (alfa = 0,05/20 = 0.0025): 4.9%
```

Capítulo 5 — Testes de Hipótese em Regressão

5.1. A lógica de um teste de hipótese

Todo teste de hipótese em econometria segue a mesma estrutura lógica, emprestada da estatística clássica (Neyman-Pearson): formula-se uma **hipótese nula** (H_0), tipicamente representando "nenhum efeito" ou "o status quo"; formula-se uma **hipótese alternativa** (H_1); calcula-se uma estatística de teste a partir dos dados; e compara-se essa estatística (ou o p-valor associado) contra um limiar de significância pré-definido, geralmente $\alpha = 0,05$.

Em regressão, a hipótese nula mais comum é $H_0 : \beta_j = 0$ — a variável x_j não tem efeito sobre y. Rejeitar H_0 não "prova" que $\beta_j \neq 0$ no sentido matemático absoluto — é uma afirmação probabilística: "os dados observados seriam muito improváveis se β_j realmente fosse zero".

5.2. Erro Tipo I e Tipo II

	H_0 verdadeira	H_0 falsa
Rejeita H_0	Erro Tipo I (falso positivo) — probabilidade α	Decisão correta

	H_0 verdadeira	H_0 falsa
Não rejeita H_0	Decisão correta	Erro Tipo II (falso negativo) — probabilidade β

Escolher $\alpha = 0,05$ significa aceitar, por convenção, uma chance de 5% de rejeitar H_0 quando ela é, na verdade, verdadeira. Reduzir α (para "0,01", por exemplo) torna o teste mais "conservador" — reduz o erro Tipo I, mas aumenta o erro Tipo II (menos poder para detectar efeitos verdadeiros pequenos). Não existe almoço grátis: a escolha de α é uma escolha sobre qual tipo de erro você está mais disposto a tolerar, algo que deveria depender do contexto da pergunta (um teste clínico de segurança de medicamento tolera erro Tipo I muito menor que um teste exploratório em pesquisa econômica).

5.3. Teste t vs. teste F — quando usar cada um

Situação	Teste apropriado
"A variável `desemprego`, isoladamente, afeta a inflação?"	Teste t sobre $\hat{\beta}_{\text{desemprego}}$
"As variáveis `desemprego` e `câmbio`, **juntas**, melhoram o modelo em relação a um modelo só com `ipca_lag`?"	Teste F de restrições conjuntas (comparação de modelos aninhados)
"Todos os coeficientes do modelo são conjuntamente zero?"	Teste F reportado no `summary()`

O teste F para restrições conjuntas (não apenas "todos os betas são zero", mas um subconjunto específico) pode ser calculado comparando um **modelo restrito** (sem as variáveis de interesse) contra o **modelo irrestrito** (com elas):

$$F = \frac{(\text{SQR}_r - \text{SQR}_{\text{ir}})/q}{\text{SQR}_{\text{ir}}/(n-k-1)}$$

onde q é o número de restrições testadas, SQR_r e SQR_{ir} são as somas dos quadrados dos resíduos dos modelos restrito e irrestrito, respectivamente.

```
modelo_irrestrito = smf.ols("ipca ~ desemprego + ipca_lag1 + cambio", data=df).fit()
modelo_restrito = smf.ols("ipca ~ ipca_lag1", data=df).fit()

# statsmodels tem um método pronto para o teste F de restrições
f_test = modelo_irrestrito.compare_f_test(modelo_restrito)
print(f"F-statistic: {f_test[0]:.4f}, p-valor: {f_test[1]:.4f}")
```

5.4. O debate sobre o limiar de "0,05"

Vale um comentário honesto: o limiar $\alpha = 0,05$ é uma convenção histórica (popularizada por Ronald Fisher nos anos 1920), não uma lei da natureza. A American Statistical Association publicou, em 2016, um posicionamento oficial alertando contra o uso mecânico de "p < 0,05 = significativo, p > 0,05 = não significativo" como critério binário de "verdade" — um efeito com $p = 0,06$ não é qualitativamente diferente de um com $p = 0,04$; é uma diferença de grau, não de natureza. Em pesquisa econômica aplicada, é considerada boa prática reportar o p-valor exato (não apenas "significativo/não significativo") e discutir a **magnitude econômica** do efeito (o coeficiente em si), não apenas sua significância estatística — um efeito pode ser estatisticamente significativo e economicamente irrelevante (amostras muito grandes detectam como "significativo" até efeitos triviais).

5.5. Exemplo Resolvido — Testando a Curva de Phillips

Enunciado: a Curva de Phillips original propõe uma relação **negativa** entre desemprego e inflação. Rode a regressão abaixo com dados reais do BCB e teste formalmente a hipótese $H_0 : \beta_{\text{desemprego}} \geq 0$ contra $H_1 : \beta_{\text{desemprego}} < 0$ (teste unilateral).

```
ipca = sgs.get({"ipca": 433}, start="2012-01-01")
desemprego = sgs.get({"desemprego": 24369}, start="2012-01-01")
```

```
df = pd.merge(ipca, desemprego, on="Date")
df["ipca_lag1"] = df["ipca"].shift(1)
df = df.dropna()

X = sm.add_constant(df[["desemprego", "ipca_lag1"]])
y = df["ipca"]
modelo = sm.OLS(y, X).fit()
```

Solução comentada:

```
coef = modelo.params["desemprego"]
p_bilateral = modelo.pvalues["desemprego"]

print(f"Coeficiente desemprego: {coef:.4f}")
print(f"p-valor (bilateral, reportado pelo statsmodels): {p_bilateral:.4f}")

# Para um teste UNILATERAL (H1: beta < 0), quando o coeficiente
# estimado tem o sinal esperado (negativo), o p-valor unilateral
# é METADE do p-valor bilateral reportado pelo summary()
if coef < 0:
    p_unilateral = p_bilateral / 2
    print(f"p-valor (unilateral): {p_unilateral:.4f}")
else:
    print("Coeficiente com sinal contrário ao esperado — não rejeita H0 de forma alguma")
```

O ponto conceitual central: o `summary()` do statsmodels sempre reporta o p-valor de um teste **bilateral** ($H_1 : \beta \neq 0$), porque é o caso mais genérico e conservador. Quando a teoria econômica prevê um sinal específico (aqui, negativo, conforme a Curva de Phillips original) e o coeficiente estimado tem, de fato, esse sinal, o pesquisador pode legitimamente usar um teste **unilateral**, que divide o p-valor por dois (fica mais fácil rejeitar H_0, porque toda a "massa" de rejeição está concentrada de um lado da distribuição). O cuidado necessário: essa divisão só é válida se o sinal do coeficiente **já era esperado antes de ver os dados** — decidir "agora que vi que deu negativo, vou fazer o teste unilateral" depois de observar o resultado é uma forma de viés de confirmação estatística (às vezes chamado informalmente de "p-hacking").

> **Aprofundando a teoria — o significado de "mantendo as demais constantes": o teorema de Frisch-Waugh-Lovell**
> Em $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon$, o coeficiente β_1 é o efeito de x_1 *mantido* x_2 *constante*. O teorema de Frisch-Waugh-Lovell mostra o que isso significa: $\hat{\beta}_1$ pode ser obtido em três passos — (1) regredir y em x_2 e guardar os resíduos (a parte de y não explicada por x_2); (2) regredir x_1 em x_2 e guardar os resíduos (a parte de x_1 não explicada por x_2); (3) regredir um resíduo no outro. O coeficiente é o mesmo da regressão múltipla. Ou seja, a regressão múltipla "limpa" cada variável da influência das outras e usa só a variação restante:

```
rng_f = np.random.default_rng(8)
n_f = 500
escolaridade = rng_f.normal(12, 3, n_f)
experiencia = 0.5 * escolaridade + rng_f.normal(10, 4, n_f)                #
correlacionada com escolaridade
log_salario = 0.5 + 0.09 * escolaridade + 0.03 * experiencia + rng_f.normal(0, 0.3, n_f)

multipla = sm.OLS(log_salario, sm.add_constant(np.column_stack([escolaridade,
experiencia]))).fit()
res_y = sm.OLS(log_salario, sm.add_constant(experiencia)).fit().resid         # passo 1
res_x = sm.OLS(escolaridade, sm.add_constant(experiencia)).fit().resid        # passo 2
fwl = sm.OLS(res_y, res_x).fit()                                               # passo 3
simples = sm.OLS(log_salario, sm.add_constant(escolaridade)).fit()
```

```
print(f"Coeficiente de escolaridade -> múltipla: {multipla.params[1]:.5f} | Frisch-Waugh-
Lovell: {fwl.params[0]:.5f} (idênticos)")
print(f"Regressão simples (sem controlar experiência): {simples.params[1]:.4f} |
verdadeiro: 0.09 | correlação escolaridade-experiência: {np.corrcoef(escolaridade,
experiencia)[0, 1]:.2f}")
print(f"R² múltipla: {multipla.rsquared:.3f} | R² ajustado: {multipla.rsquared_adj:.3f}")
```

```
Coeficiente de escolaridade -> múltipla: 0.08768 | Frisch-Waugh-Lovell: 0.08768 (idênticos)
Regressão simples (sem controlar experiência): 0.1030 | verdadeiro: 0.09 | correlação
escolaridade-experiência: 0.39
R² múltipla: 0.555 | R² ajustado: 0.553
```

Capítulo 6 — Regressão Múltipla

6.1. Por que adicionar mais variáveis

O modelo de regressão múltipla generaliza o simples para k variáveis explicativas:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \ldots + \beta_k x_{\text{ki}} + u_i$$

A motivação principal não é apenas "melhorar o R^2" — é **controlar por outros fatores** que poderiam confundir a relação de interesse, aproximando-se do que a premissa de exogeneidade (Capítulo 3) exige. Cada $\hat{\beta}_j$ passa a ser interpretado **ceteris paribus** — "mantendo todas as outras variáveis do modelo constantes" — uma expressão em latim que aparece o tempo todo em econometria e que vale memorizar precisamente: sem ela, a interpretação de um coeficiente múltiplo perde o sentido.

```
df["poupanca"] = df["renda"] - df["consumo"]
df["juros"] = 5 + np.random.normal(0, 1, len(df))

X = sm.add_constant(df[["renda", "juros", "poupanca"]])
y = df["consumo"]

modelo = sm.OLS(y, X).fit()
print(modelo.summary())
```

6.2. R² ajustado como critério de comparação

Como visto no Capítulo 4.2, use sempre o R^2 **ajustado** (`modelo.rsquared_adj`), nunca o R^2 simples, para decidir se vale a pena manter uma variável adicional no modelo — ou, melhor ainda, apoie-se em teoria econômica (a variável tem uma justificativa causal para estar no modelo?) mais do que em uma busca puramente estatística por "o que aumenta o R^2".

6.3. Exemplo Resolvido — Comparando regressão simples e múltipla

Enunciado: usando os dados de consumo, renda, juros e poupança do item 6.1, compare o $\hat{\beta}_{\text{renda}}$ estimado na regressão simples (Capítulo 2.3, apenas `renda`) com o estimado na regressão múltipla (com `juros` e `poupanca` adicionados). Eles são iguais? Por que isso é ou não é esperado?

Solução comentada:

```
simples = sm.OLS(df["consumo"], sm.add_constant(df["renda"])).fit()
multipla = sm.OLS(df["consumo"], sm.add_constant(df[["renda", "juros", "poupanca"]])).fit()

print(f"Beta_renda (simples):  {simples.params['renda']:.4f}")
print(f"Beta_renda (múltipla): {multipla.params['renda']:.4f}")
```

Em geral, **não** são exatamente iguais — e essa é a regra, não a exceção. O coeficiente de `renda` muda entre as duas especificações na exata medida em que `renda` é correlacionada com as variáveis adicionadas (`juros`, `poupanca`) **e** essas variáveis adicionadas têm efeito próprio sobre `consumo`. Neste exemplo específico, `poupanca` foi construída como `renda - consumo` — ou seja, é quase uma transformação determinística das próprias variáveis já no modelo, o que tende a inflar a colinearidade (assunto do próximo capítulo) e tornar os coeficientes individualmente instáveis, mesmo que o ajuste conjunto do modelo pareça bom. Isso ilustra, de forma propositalmente exagerada, por que a escolha de quais variáveis incluir numa regressão múltipla é uma decisão teórica, não apenas mecânica.

Aprofundando a teoria — a fórmula do viés de variável omitida
Suponha o modelo verdadeiro $y = \beta_0 + \beta_1 x + \beta_2 z + \varepsilon$, mas o analista omite z e regride y só em x. O coeficiente estimado converge para

$$\hat{\beta}_1^{\text{curto}} \to \beta_1 + \beta_2 \cdot \frac{\text{Cov}(x, z)}{\text{Var}(x)}$$

O segundo termo é o **viés**: o efeito omitido (β_2) multiplicado pela relação entre a variável incluída e a omitida ($\delta = \frac{\text{Cov}(x,z)}{\text{Var}(x)}$). O sinal do viés é o produto dos sinais de β_2 e de δ; o viés some se z não afeta y ($\beta_2 = 0$) *ou* se z não se relaciona com x ($\delta = 0$). Simulação, com $\beta_1 = 2$, $\beta_2 = 3$ e correlação entre x e z:

```
rng_v = np.random.default_rng(4)
n_v = 5000
z = rng_v.normal(size=n_v)
x = 0.6 * z + rng_v.normal(scale=0.8, size=n_v)
y = 1 + 2 * x + 3 * z + rng_v.normal(size=n_v)
curta = sm.OLS(y, sm.add_constant(x)).fit()
longa = sm.OLS(y, sm.add_constant(np.column_stack([x, z]))).fit()
delta = np.cov(x, z)[0, 1] / np.var(x, ddof=1)
print(f"Modelo CURTO (omite z): b1 = {curta.params[1]:.3f} | modelo COMPLETO: b1 =
{longa.params[1]:.3f} | verdadeiro: 2")
print(f"Viés previsto = b2 * Cov(x,z)/Var(x) = 3 x {delta:.3f} = {3 * delta:.3f} | viés
observado = {curta.params[1] - 2:.3f}")
```

```
Modelo CURTO (omite z): b1 = 3.757 | modelo COMPLETO: b1 = 1.987 | verdadeiro: 2
Viés previsto = b2 * Cov(x,z)/Var(x) = 3 x 0.591 = 1.774 | viés observado = 1.757
```

Exemplo resolvido — causalidade reversa e simultaneidade
A regressão de gastos com segurança sobre criminalidade pode dar um coeficiente *positivo* ("mais policiais, mais crime") porque a causalidade corre nos dois sentidos: o crime *atrai* gastos. Quando y também afeta x, o regressor fica correlacionado com o erro e o OLS é **viesado e inconsistente** (violação da exogeneidade). Soluções vão além do OLS (variáveis instrumentais, experimentos naturais, diferenças em diferenças) e exigem pensar no *desenho*, não só no estimador.

Capítulo 7 — Correlação Não é Causalidade

7.1. Por que este é o capítulo mais importante da apostila

Se há um único conceito que um curso de econometria precisa deixar gravado, é este: **um coeficiente de regressão estatisticamente significativo não implica, por si só, uma relação causal**. Essa afirmação parece óbvia quando enunciada assim, mas é sistematicamente esquecida na prática — inclusive por pesquisadores experientes — porque o software não distingue: `sm.OLS(y, X).fit()` roda exatamente igual, produz um `summary()` igualmente "sério", esteja ou não a relação estimada sendo causal.

O exemplo didático clássico (repetido em praticamente todo curso introdutório) é a correlação positiva entre venda de sorvetes e número de afogamentos em praias. Ninguém defende que sorvete causa afogamento — ambos são causados por uma terceira variável, a temperatura (dias quentes aumentam tanto o consumo de sorvete quanto a frequência de banhistas, logo o número de afogamentos). Essa terceira variável é chamada de **variável de confusão** (*confounder*), e sua omissão do modelo é a causa mais comum de correlação espúria.

7.2. Viés de variável omitida (Omitted Variable Bias)

Formalmente: suponha que o modelo verdadeiro seja

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + u$$

mas você estima, por falta de dados ou desconhecimento, apenas

$$y = \beta_0 + \tilde{\beta}_1 x_1 + v$$

Se x_2 (a variável omitida) está correlacionada com x_1 **e** afeta y (isto é, $\beta_2 \neq 0$), então $\tilde{\beta}_1$ é um estimador **viesado** de β_1 — o viés não desaparece com mais dados; ele é uma propriedade estrutural da má especificação do modelo. A direção do viés segue uma regra prática:

$$\text{Viés} = \beta_2 \cdot \frac{\text{Cov}(x_1, x_2)}{\text{Var}(x_1)}$$

Ou seja: o viés é positivo quando β_2 e a correlação entre x_1 e x_2 têm o **mesmo sinal**, e negativo quando têm sinais opostos.

Exemplo econômico clássico: estimar o retorno da educação sobre o salário (equação de Mincer, ver Capítulo 10) omitindo a **habilidade inata** do indivíduo. Pessoas mais habilidosas tendem a estudar mais (correlação positiva entre `escolaridade` e `habilidade`) e a habilidade, por si só, também eleva o salário — logo, uma regressão simples de `salario` contra `escolaridade` tende a **superestimar** o verdadeiro retorno causal da educação, atribuindo a ela parte do efeito que, na verdade, vem da habilidade inata (não observada, e por isso frequentemente chamada de "viés de habilidade" na literatura de economia do trabalho).

7.3. Causalidade reversa

Outro mecanismo que gera correlação sem causalidade (na direção assumida) é a **causalidade reversa**: x parece afetar y, mas na verdade é y que afeta x, ou os dois se afetam mutuamente. Exemplo clássico em macroeconomia: uma regressão simples de crescimento do PIB contra gastos públicos em saúde pode mostrar correlação positiva — mas países mais ricos (PIB per capita mais alto) também têm mais recursos para gastar em saúde, então parte da correlação vem do PIB **causando** mais gasto em saúde, não o oposto. Distinguir as duas direções exige desenho de pesquisa (dados em painel com defasagens, variáveis instrumentais, experimentos naturais) — não é algo que uma única regressão de corte transversal resolve.

7.4. O padrão-ouro: desenho experimental vs. dados observacionais

	Experimento controlado	Dados observacionais
Atribuição do tratamento	Aleatória (o pesquisador decide quem recebe o "tratamento")	Não aleatória (as próprias pessoas/empresas/países "escolhem" seu valor de x)
Variáveis de confusão	Balanceadas entre grupos por construção (a aleatorização "quebra" a correlação entre x e possíveis confounders)	Podem estar presentes e não observadas
Exemplo em economia	Testes A/B de política pública (ex: transferência de renda condicionada, sorteada por loteria entre elegíveis)	A maioria dos dados macro/micro disponíveis (PIB, inflação, salários)

Como experimentos controlados são raros e caros em economia, a disciplina desenvolveu métodos para se aproximar da lógica experimental usando dados observacionais — variáveis instrumentais (IV), diferenças-em-diferenças (DiD), descontinuidade de regressão (RDD) — que ficam fora do escopo desta apostila introdutória,

mas que valem menção porque são a resposta metodológica ao problema deste capítulo, não à correlação simples.

7.5. Exemplo Resolvido — Identificando um confounder plausível

Enunciado: uma regressão simples mostra correlação positiva e estatisticamente significativa entre `numero_de_bancos_centrais_no_pais` (sempre 1, então descartado) — na verdade, entre `gasto_militar_per_capita` e `expectativa_de_vida` entre países. Antes de concluir que gasto militar aumenta expectativa de vida, que variável de confusão óbvia deveria ser investigada, e por quê?

Solução comentada:

O candidato mais óbvio é o **PIB per capita** (ou, de forma equivalente, o nível de desenvolvimento econômico do país). Países mais ricos tendem a ter tanto orçamento maior para gastos públicos em geral (incluindo militar) quanto melhor infraestrutura de saúde, saneamento e nutrição — o que eleva diretamente a expectativa de vida, independentemente do gasto militar em si. A correlação observada entre gasto militar e expectativa de vida é plausivelmente **espúria**, no sentido de ser inteiramente (ou majoritariamente) explicada pelo PIB per capita como variável de confusão comum.

O teste correto **não é** simplesmente "adicionar PIB per capita e ver se o coeficiente de gasto militar continua significativo" (embora seja um primeiro passo razoável) — é reconhecer que, mesmo controlando por PIB, poderiam existir outras variáveis de confusão não observadas (qualidade institucional, por exemplo), e que a ausência de um desenho experimental (Capítulo 7.4) significa que **nenhuma regressão observacional, por si só, estabelece causalidade com certeza** — apenas fornece evidência mais ou menos consistente com uma hipótese causal, a depender de quão bem controlados estão os confounders plausíveis.

```
# Passo mínimo de investigação: comparar coeficiente com e sem o confounder
simples = smf.ols("expectativa_vida ~ gasto_militar_per_capita", data=df).fit()
controlado = smf.ols("expectativa_vida ~ gasto_militar_per_capita + pib_per_capita",
data=df).fit()

print(f"Sem controle:  {simples.params['gasto_militar_per_capita']:.4f}")
print(f"Com controle:  {controlado.params['gasto_militar_per_capita']:.4f}")
# Se o coeficiente cair para perto de zero (ou perder significância) ao
# controlar por PIB, isso reforça a hipótese de que a correlação original
# era espúria, mediada pelo PIB per capita.
```

> **Aprofundando a teoria — VIF e a inflação do erro-padrão**
> A multicolinearidade **não viesa** os coeficientes; ela **infla seus erros-padrão**. O fator de inflação da variância do regressor j é $\text{VIF}_j = \frac{1}{1-R_j^2}$, onde R_j^2 vem da regressão de x_j nos demais regressores; então $\text{ep}\left(\hat{\beta}_j\right) \approx \sqrt{\text{VIF}_j} \times \text{ep}_{\text{sem colinearidade}}$. Um VIF de 10 multiplica o erro-padrão por $\sqrt{10} \approx 3,2$. O experimento varia a correlação entre dois regressores de 0 a "0,99":

```
rng_m = np.random.default_rng(6)
n_m = 400
print(f"{'corr(x1,x2)':>12} {'VIF':>7} {'b1 (média)':>11} {'ep(b1) médio':>13}")
for rho in (0.0, 0.5, 0.9, 0.99):
    b_s, ep_s = [], []
    for _ in range(600):
        x1 = rng_m.normal(size=n_m)
        x2 = rho * x1 + np.sqrt(1 - rho ** 2) * rng_m.normal(size=n_m)
        y = 1 + 1.0 * x1 + 1.0 * x2 + rng_m.normal(size=n_m)
        ajuste = sm.OLS(y, sm.add_constant(np.column_stack([x1, x2]))).fit()
        b_s.append(ajuste.params[1]); ep_s.append(ajuste.bse[1])
    print(f"{rho:>12.2f} {1 / (1 - rho ** 2):>7.1f} {np.mean(b_s):>11.3f}
{np.mean(ep_s):>13.4f}")
```

```
corr(x1,x2)     VIF  b1 (média)  ep(b1) médio
       0.00     1.0       1.000        0.0502
       0.50     1.3       0.998        0.0581
       0.90     5.3       1.002        0.1149
       0.99    50.3       1.009        0.3549
```

Intuição
A média de $\hat{\beta}_1$ continua em "1,0" (sem viés), mas o erro-padrão explode quando as variáveis se confundem: o modelo "não sabe" qual das duas é responsável pelo efeito. É um problema de *informação insuficiente*, não de erro de método: a saída é mais dados, combinar as variáveis (índice) ou aceitar que só o efeito *conjunto* é identificado.

Capítulo 8 — Multicolinearidade

8.1. O que é e por que é um problema

Multicolinearidade ocorre quando duas ou mais variáveis explicativas estão fortemente correlacionadas entre si. Tecnicamente, ela **não** viola nenhuma das premissas de Gauss-Markov necessárias para que o OLS seja não viesado (desde que não haja colinearidade **perfeita** — premissa 3) — mas ela infla drasticamente a **variância** dos coeficientes estimados, tornando-os imprecisos e instáveis: pequenas mudanças na amostra podem alterar substancialmente $\hat{\beta}_1$ e $\hat{\beta}_2$, e às vezes até seus sinais.

A intuição: se `renda` e `poupanca` são quase colineares (como no exemplo do Capítulo 6.3), o modelo tem dificuldade em "separar" o efeito de uma da outra — geometricamente, as duas variáveis se movem quase juntas, então não há variação independente suficiente em cada uma para estimar seu efeito isolado com precisão.

8.2. Sintomas e o VIF (Variance Inflation Factor)

Um sintoma clássico: coeficientes individuais não significativos no teste t (Capítulo 5), mas o teste F conjunto fortemente significativo — o modelo como um todo explica bem y, mas o software "não sabe" a qual variável atribuir esse poder explicativo.

O diagnóstico formal é o **Fator de Inflação da Variância** (VIF), calculado para cada variável explicativa regredindo-a contra todas as outras:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

onde R_j^2 é o R^2 de uma regressão auxiliar de x_j contra todas as demais variáveis explicativas do modelo. Um $\text{VIF}_j = 1$ indica ausência de colinearidade; a regra prática mais citada na literatura é que $\text{VIF} > 10$ (equivalente a $R_j^2 > 0,9$ na regressão auxiliar) sinaliza multicolinearidade preocupante, embora alguns autores usem o limiar mais conservador de 5.

```python
from statsmodels.stats.outliers_influence import variance_inflation_factor

X = sm.add_constant(df[["renda", "juros", "poupanca"]])
vif = pd.DataFrame()
vif["variavel"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif)
```

8.3. O que fazer diante de multicolinearidade

Estratégia	Quando usar
Remover uma das variáveis redundantes	Quando as duas variáveis medem, na prática, o mesmo conceito
Combinar as variáveis num índice	Quando ambas são dimensões legítimas de um mesmo construto (ex: índice de qualidade institucional)
Aumentar o tamanho da amostra	Reduz a variância dos coeficientes mesmo sem resolver a colinearidade em si
Aceitar e reportar	Quando o objetivo é apenas **prever** y (não interpretar β_j individualmente) — multicolinearidade não prejudica a previsão do modelo como um todo

O último item merece destaque: se o objetivo do modelo é puramente preditivo (o valor de $\hat{y}$ importa, não a interpretação de cada $\hat{\beta}_j$ isoladamente), multicolinearidade é um problema bem menos grave — é precisamente quando se quer **interpretar** cada coeficiente separadamente que ela se torna uma preocupação central.

8.4. Exemplo Resolvido — Diagnosticando e corrigindo multicolinearidade

Enunciado: você suspeita que `juros` e `taxa_selic_media` (duas variáveis do seu dataset) sejam quase a mesma coisa medida de formas ligeiramente diferentes. Calcule o VIF de um modelo que inclui as duas e decida o que fazer.

Solução comentada:

```
np.random.seed(0)
n = 100
juros = np.random.normal(10, 2, n)
taxa_selic_media = juros + np.random.normal(0, 0.1, n)  # quase idêntica a "juros"
y_sim = 100 + 2 * juros + np.random.normal(0, 5, n)

df_vif = pd.DataFrame({"juros": juros, "taxa_selic_media": taxa_selic_media, "y": y_sim})
X = sm.add_constant(df_vif[["juros", "taxa_selic_media"]])

vif = pd.DataFrame()
vif["variavel"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif)
# VIF de "juros" e "taxa_selic_media" tipicamente na casa de centenas —
# muito acima do limiar de 10

modelo_colinear = sm.OLS(df_vif["y"], X).fit()
print(modelo_colinear.pvalues)  # provavelmente nenhum dos dois é individualmente
significativo

# Correção: manter apenas uma das duas variáveis
X_corrigido = sm.add_constant(df_vif[["juros"]])
modelo_corrigido = sm.OLS(df_vif["y"], X_corrigido).fit()
print(modelo_corrigido.pvalues)  # agora "juros" é claramente significativo
```

O VIF confirma a suspeita (valores muito acima de 10), e o `summary()` do modelo com as duas variáveis mostra exatamente o sintoma descrito no item 8.2: nenhum dos dois coeficientes é individualmente significativo, apesar de existir, de fato, uma relação forte entre `juros` e `y`. Ao remover a variável redundante, o coeficiente restante recupera significância — a informação estava lá o tempo todo, apenas "diluída" entre duas variáveis que carregavam essencialmente o mesmo conteúdo informacional.

Aprofundando a teoria — por que os erros-padrão robustos importam: cobertura do intervalo de confiança
Com heterocedasticidade ($\mathrm{Var}(\varepsilon \mid x)$ varia com x), os coeficientes OLS continuam **não viesados**, mas a fórmula usual do erro-padrão fica **errada**, e os intervalos de confiança não cobrem o valor verdadeiro com a probabilidade anunciada. O erro-padrão robusto (White/HC) corrige. Para medir, simulamos 3.000 amostras com $\mathrm{Var}(\varepsilon) \propto x^2$ e contamos quantas vezes o intervalo de 95% *contém* o verdadeiro $\beta_1 = 1$ (a "cobertura" deveria ser 95%):

```
rng_h = np.random.default_rng(12)
n_h, reps_h = 100, 3000
cobre_classico = cobre_robusto = 0
for _ in range(reps_h):
    x = rng_h.uniform(1, 10, n_h)
    y = 1 + 1.0 * x + rng_h.normal(0, 0.5 * x)                          # desvio padrão do
erro cresce com x
    ajuste = sm.OLS(y, sm.add_constant(x)).fit()
    robusto = ajuste.get_robustcov_results(cov_type="HC3")
    cobre_classico += ajuste.conf_int()[1, 0] <= 1.0 <= ajuste.conf_int()[1, 1]
    cobre_robusto += robusto.conf_int()[1, 0] <= 1.0 <= robusto.conf_int()[1, 1]
print(f"Cobertura do IC de 95% com erro-padrão clássico: {cobre_classico / reps_h:.1%}")
print(f"Cobertura do IC de 95% com erro-padrão robusto (HC3): {cobre_robusto /
reps_h:.1%}")
```

```
Cobertura do IC de 95% com erro-padrão clássico: 93.3%
Cobertura do IC de 95% com erro-padrão robusto (HC3): 95.2%
```

Intuição
Com o erro-padrão clássico o intervalo "de 95%" cobre o valor verdadeiro em cerca de 93% das amostras: o nível anunciado não é o real. Com o robusto (HC3), a cobertura volta a ficar próxima de 95%. A diferença parece pequena aqui, mas cresce com a intensidade da heterocedasticidade e com a presença de pontos de alavanca.

Atenção — erro comum
Erros-padrão robustos **não corrigem viés**, só a inferência. E há quem recomende usá-los *sempre* em trabalho aplicado, porque quando a variância é constante eles perdem pouco, e quando não é, evitam conclusões erradas. O custo de usar o erro-padrão clássico "por padrão" sem checar é maior que o de usar o robusto sem necessidade.

Capítulo 9 — Heterocedasticidade

9.1. Revisitando o conceito

Já introduzida no Capítulo 3.4: heterocedasticidade é a violação da premissa de que a variância do erro é constante ($\mathrm{Var}(u|x) = \sigma^2$ para todo x). O nome vem do grego (*hetero* = diferente, *skedasis* = dispersão) — o oposto, variância constante, é a **homocedasticidade**.

9.2. Consequência: coeficientes certos, confiança errada

Vale repetir, porque é frequentemente mal entendido: heterocedasticidade **não** torna $\hat{\beta}$ viesado — o OLS continua sendo não viesado. O que ela invalida são os erros-padrão calculados pela fórmula usual do OLS, e por consequência os testes t, F e os intervalos de confiança. Isso significa que um pesquisador pode, sem saber, reportar uma variável como "estatisticamente significativa" (ou "não significativa") quando o erro-padrão correto diria o contrário.

9.3. Testando heterocedasticidade — Breusch-Pagan e White

O **teste de Breusch-Pagan** regride o quadrado dos resíduos ($\hat{u}_i^2$) contra as variáveis explicativas do modelo original, testando se essas variáveis ajudam a explicar a magnitude do erro:

$$H_0 : \text{homocedasticidade} \quad \text{vs.} \quad H_1 : \text{heterocedasticidade}$$

```
from statsmodels.stats.diagnostic import het_breuschpagan

residuos = modelo.resid
bp_test = het_breuschpagan(residuos, modelo.model.exog)
labels = ["LM Statistic", "LM p-valor", "F Statistic", "F p-valor"]
print(dict(zip(labels, bp_test)))
# Se p > 0.05: não rejeita H0 (homocedástico)
# Se p < 0.05: rejeita H0 (heterocedástico)
```

O **teste de White** é uma generalização mais flexível (inclui termos quadráticos e produtos cruzados das variáveis explicativas), com a vantagem de não exigir suposição sobre a forma funcional da heterocedasticidade, mas com o custo de menor poder estatístico em amostras pequenas (mais parâmetros estimados na regressão auxiliar).

9.4. Correção: erros-padrão robustos

A solução mais usada na prática **não** é "consertar" o modelo, mas recalcular os erros-padrão de forma que continuem válidos mesmo sob heterocedasticidade — os chamados **erros-padrão robustos** (ou "erros-padrão de White", ou HC — *Heteroskedasticity-Consistent*). Os coeficientes estimados ($\hat{\beta}$) permanecem exatamente os mesmos; apenas os erros-padrão, e por consequência os testes t/F e intervalos de confiança, mudam.

```
# Erros-padrão robustos a heterocedasticidade (HC1 é uma correção comum)
modelo_robusto = sm.OLS(y, X).fit(cov_type="HC1")
print(modelo_robusto.summary())

# Compare os erros-padrão "normais" com os robustos
print("Normal:  ", modelo.bse.values)
print("Robusto: ", modelo_robusto.bse.values)
```

Em pesquisa econômica aplicada moderna, é considerada boa prática usar erros-padrão robustos **por padrão** em dados de corte transversal, mesmo sem um teste formal de heterocedasticidade prévio — o custo de usá-los quando não são necessários é pequeno (os erros-padrão robustos convergem para os usuais quando a homocedasticidade de fato vale), enquanto o custo de não usá-los quando são necessários pode ser uma inferência inteiramente equivocada.

9.5. Exemplo Resolvido — Heterocedasticidade no gasto com alimentação

Enunciado: retomando o cenário do Capítulo 3.4 (gasto com alimentação vs. renda, com variância do erro crescente na renda), simule os dados, teste formalmente a heterocedasticidade com Breusch-Pagan, e compare os erros-padrão usuais com os robustos.

Solução comentada:

```
np.random.seed(1)
n = 300
renda_fam = np.random.uniform(1000, 20000, n)
# erro com variancia crescente na renda -> heterocedasticidade por construcao
erro = np.random.normal(0, 1, n) * (renda_fam / 2000)
gasto_alimentacao = 300 + 0.15 * renda_fam + erro

df_hetero = pd.DataFrame({"renda": renda_fam, "gasto": gasto_alimentacao})
X = sm.add_constant(df_hetero["renda"])
y = df_hetero["gasto"]
```

```
modelo = sm.OLS(y, X).fit()

bp_test = het_breuschpagan(modelo.resid, modelo.model.exog)
print(f"Breusch-Pagan p-valor: {bp_test[1]:.6f}")  # tipicamente muito < 0.05

modelo_robusto = sm.OLS(y, X).fit(cov_type="HC1")

print(f"SE(renda) usual:   {modelo.bse['renda']:.5f}")
print(f"SE(renda) robusto:  {modelo_robusto.bse['renda']:.5f}")
```

O teste de Breusch-Pagan rejeita fortemente a homocedasticidade (p-valor muito abaixo de "0,05"), confirmando o padrão de "leque" esperado por construção dos dados. Nesse tipo de heterocedasticidade (variância crescendo com o nível de x), é comum que o erro-padrão robusto seja **maior** que o usual — o que significa que o modelo OLS "ingênuo" estava **superestimando a precisão** do coeficiente de `renda`, um erro que, sem o teste e a correção, passaria despercebido no `summary()` padrão.

> **Aprofundando a teoria — a armadilha da dummy e a dummy como diferença de médias**
> Com G categorias, inclui-se $\boldsymbol{G-1}$ **dummies** mais o intercepto: incluir todas as G cria **colinearidade perfeita** (a soma das dummies é igual à coluna de 1), e a matriz X^TX deixa de ser inversível. Além disso, o coeficiente de uma dummy é simplesmente a **diferença de médias** entre o grupo e a categoria de referência: uma regressão com uma só dummy reproduz o teste t de duas amostras. A demonstração numérica:

```
rng_d = np.random.default_rng(14)
n_d = 400
mulher = rng_d.integers(0, 2, n_d)
log_sal = 2.0 - 0.15 * mulher + rng_d.normal(0, 0.4, n_d)
df_d = pd.DataFrame({"log_sal": log_sal, "mulher": mulher, "homem": 1 - mulher})
ajuste_d = sm.OLS(df_d["log_sal"], sm.add_constant(df_d[["mulher"]])).fit()
dif_medias = df_d.loc[df_d.mulher == 1, "log_sal"].mean() - df_d.loc[df_d.mulher == 0,
"log_sal"].mean()
t_welch = sps.ttest_ind(df_d.loc[df_d.mulher == 1, "log_sal"], df_d.loc[df_d.mulher == 0,
"log_sal"], equal_var=True)
print(f"Coeficiente da dummy: {ajuste_d.params['mulher']:.4f} | diferença de médias:
{dif_medias:.4f}")
print(f"t da regressão: {ajuste_d.tvalues['mulher']:.3f} | t do teste de duas amostras:
{t_welch.statistic:.3f}")

X_armadilha = sm.add_constant(df_d[["mulher", "homem"]])                    # intercepto +
as DUAS dummies
print(f"Posto da matriz: {np.linalg.matrix_rank(X_armadilha.values)} de
{X_armadilha.shape[1]} colunas -> colinearidade perfeita")
print(f"Efeito percentual exato da dummy em log-linear: {(np.exp(ajuste_d.params['mulher'])
- 1) * 100:.1f}% (a aproximação seria {ajuste_d.params['mulher'] * 100:.1f}%)")
```

```
Coeficiente da dummy: -0.1739 | diferença de médias: -0.1739
t da regressão: -4.398 | t do teste de duas amostras: -4.398
Posto da matriz: 2 de 3 colunas -> colinearidade perfeita
Efeito percentual exato da dummy em log-linear: -16.0% (a aproximação seria -17.4%)
```

Capítulo 10 — Variáveis Dummy e a Equação de Mincer

10.1. Codificando categorias como números

Regressão linear opera sobre números, mas muitas variáveis econômicas relevantes são categóricas: região, gênero, setor de atividade, regime cambial. A solução é a **variável dummy** (ou *indicadora*): uma variável que assume valor 1 quando uma categoria está presente, e 0 caso contrário.

```
df["dummy_sul"] = (df["regiao"] == "Sul").astype(int)
# ou, de forma automática para múltiplas categorias:
dummies = pd.get_dummies(df["regiao"], prefix="regiao", drop_first=True)
```

10.2. A armadilha da dummy (dummy trap)

Se uma variável categórica tem m categorias (por exemplo, 5 regiões do Brasil), deve-se incluir no modelo apenas $m - 1$ dummies, nunca as m — a categoria excluída se torna a **categoria de referência** (ou *baseline*), contra a qual todas as demais são comparadas. Incluir as m dummies **e** o intercepto gera colinearidade perfeita (a soma de todas as dummies é sempre igual a 1, exatamente igual à coluna de constante) — o software geralmente detecta isso e descarta uma coluna automaticamente, mas contar com esse comportamento automático é frágil; melhor especificar `drop_first=True` explicitamente, como no Capítulo 10.1.

10.3. A Equação de Mincer: retornos à educação

Um dos modelos mais replicados em economia do trabalho é a **equação de Mincer** (Jacob Mincer, 1974), que relaciona o logaritmo do salário à escolaridade e à experiência:

$$\ln(\text{salário}) = \beta_0 + \beta_1 \cdot \text{escolaridade} + \beta_2 \cdot \text{experiência} + \beta_3 \cdot \text{experiência}^2 + u$$

O uso do log do salário (em vez do salário em nível) e o termo quadrático de experiência serão explicados formalmente no Capítulo 11 — por ora, o ponto de interesse é como incorporar uma dummy de gênero para estimar o chamado "hiato salarial" (*gender wage gap*):

```
np.random.seed(7)
n = 500
escolaridade = np.random.randint(4, 18, n)
experiencia = np.random.randint(0, 40, n)
mulher = np.random.binomial(1, 0.5, n)

# Hiato salarial simulado de -15% para mulheres, ceteris paribus
log_salario = (7.0 + 0.10 * escolaridade + 0.03 * experiencia
               - 0.0004 * experiencia**2 - 0.15 * mulher
               + np.random.normal(0, 0.3, n))

df_mincer = pd.DataFrame({
    "log_salario": log_salario, "escolaridade": escolaridade,
    "experiencia": experiencia, "mulher": mulher
})

modelo_mincer = smf.ols(
    "log_salario ~ escolaridade + experiencia + I(experiencia**2) + mulher",
    data=df_mincer
).fit()
print(modelo_mincer.summary())
```

10.4. Interpretando o coeficiente de uma dummy em modelo log-linear

Quando a variável dependente está em log e o regressor é uma dummy, a interpretação exata do coeficiente **não é simplesmente "multiplicar por 100"** — para coeficientes grandes, a aproximação usual (Capítulo 11.2) tem um viés conhecido. A interpretação exata (correção de Halvorsen-Palmquist) é:

$$\%\Delta y = \left(e^{\hat{\beta}} - 1\right) \times 100$$

```
beta_mulher = modelo_mincer.params["mulher"]
efeito_aproximado = beta_mulher * 100
efeito_exato = (np.exp(beta_mulher) - 1) * 100

print(f"Efeito aproximado: {efeito_aproximado:.2f}%")
print(f"Efeito exato:       {efeito_exato:.2f}%")
```

Para $\hat{\beta}_{\text{mulher}} \approx -0,15$, a aproximação simples ($-15,0\%$) e a fórmula exata (algo em torno de $-13,9\%$) já divergem de forma perceptível — a diferença cresce com a magnitude do coeficiente, e é mais um motivo para preferir sempre a fórmula exata em variáveis dummy, cujo coeficiente costuma ser maior, em valor absoluto, do que os coeficientes de variáveis contínuas.

10.5. Interação entre dummy e variável contínua

Uma pergunta natural de pesquisa é: "o retorno da escolaridade sobre o salário é o mesmo para homens e mulheres?" Isso se testa com um **termo de interação** — o produto entre a dummy e a variável contínua:

```
modelo_interacao = smf.ols(
    "log_salario ~ escolaridade * mulher + experiencia + I(experiencia**2)",
    data=df_mincer
).fit()
print(modelo_interacao.summary())
# "escolaridade * mulher" no statsmodels expande automaticamente para
# escolaridade + mulher + escolaridade:mulher (o termo de interação)
```

O coeficiente de `escolaridade:mulher` mede a **diferença** no retorno da escolaridade entre mulheres e homens — se for estatisticamente significativo, é evidência de que a inclinação da relação escolaridade-salário difere por gênero, não apenas o intercepto (o "nível" do salário).

10.6. Exemplo Resolvido — Decompondo o hiato salarial estimado

Enunciado: usando o modelo `modelo_mincer` do item 10.3, calcule o hiato salarial exato (fórmula de Halvorsen-Palmquist) e verifique se ele é estatisticamente diferente de zero. Depois, discuta por que este número, mesmo sendo estatisticamente significativo, não deveria ser automaticamente interpretado como "discriminação" no mercado de trabalho.

Solução comentada:

```
beta = modelo_mincer.params["mulher"]
p_valor = modelo_mincer.pvalues["mulher"]
hiato_exato = (np.exp(beta) - 1) * 100

print(f"Hiato salarial estimado: {hiato_exato:.2f}%")
print(f"p-valor: {p_valor:.4f}")
```

O modelo estima um hiato de aproximadamente 14% (mulheres ganhando, em média, 14% menos que homens de mesma escolaridade e experiência), com p-valor tipicamente muito baixo dado o tamanho da amostra simulada (n=500) — estatisticamente significativo.

O ponto de discussão crítica: o coeficiente de `mulher` captura **toda** a diferença salarial entre os grupos que não é explicada por `escolaridade` e `experiência` — mas isso inclui, potencialmente, discriminação **e** qualquer outro fator não observado correlacionado com gênero e não incluído no modelo (diferenças em setor de atuação, jornada de trabalho, interrupções de carreira por licença-maternidade, negociação salarial, entre outros — a literatura de economia do trabalho sobre o tema é extensa precisamente por essa dificuldade de identificação). Isso é exatamente o problema de **viés de variável omitida** do Capítulo 7.2 aplicado a este contexto específico: o

coeficiente de uma dummy num modelo com controles limitados é, na melhor das hipóteses, um limite superior para o componente puramente discriminatório do hiato — não uma medida isolada e definitiva dele.

Exemplo resolvido — recuperando os parâmetros da equação de Mincer em dados simulados
Gera-se um salário com retorno à educação de 9% por ano, retorno à experiência de 4% (com rendimentos decrescentes) e verifica-se se a regressão recupera os valores, **e** o que ocorre se omitirmos a experiência ao quadrado:

```
rng_mi = np.random.default_rng(21)
n_mi = 3000
educ = rng_mi.integers(4, 20, n_mi)
exper = rng_mi.uniform(0, 35, n_mi)
lw = 0.8 + 0.09 * educ + 0.04 * exper - 0.0006 * exper ** 2 + rng_mi.normal(0, 0.35, n_mi)
completo = sm.OLS(lw, sm.add_constant(np.column_stack([educ, exper, exper ** 2]))).fit()
sem_quad = sm.OLS(lw, sm.add_constant(np.column_stack([educ, exper]))).fit()
print("Verdadeiros:  educ = 0.090 | exper = 0.0400 | exper² = -0.00060")
print(f"Mincer completo: educ = {completo.params[1]:.3f} | exper = {completo.params[2]:.4f}
| exper² = {completo.params[3]:.5f} | R² = {completo.rsquared:.3f}")
print(f"Sem exper²     : educ = {sem_quad.params[1]:.3f} | exper = {sem_quad.params[2]:.4f}
| R² = {sem_quad.rsquared:.3f}")
pico = -completo.params[2] / (2 * completo.params[3])
print(f"Experiência em que o salário para de crescer (pico): {pico:.1f} anos")
```

```
Verdadeiros:  educ = 0.090 | exper = 0.0400 | exper² = -0.00060
Mincer completo: educ = 0.091 | exper = 0.0391 | exper² = -0.00055 | R² = 0.643
Sem exper²     : educ = 0.091 | exper = 0.0198 | R² = 0.636
Experiência em que o salário para de crescer (pico): 35.6 anos
```

Aprofundando a teoria — as quatro formas log-nível: um quadro numérico
Para um aumento no regressor x, o efeito em y depende da forma funcional:

Modelo	Forma	Interpretação de β
Nível-nível	$y = \beta_0 + \beta_1 x$	+1 unidade de x muda y em β_1 unidades
Log-nível	$\ln y = \beta_0 + \beta_1 x$	+1 unidade de x muda y em aproximadamente $100\beta_1\%$
Nível-log	$y = \beta_0 + \beta_1 \ln x$	+1% em x muda y em aproximadamente $\beta_1/100$ unidades
Log-log	$\ln y = \beta_0 + \beta_1 \ln x$	+1% em x muda y em $\beta_1\%$ (**elasticidade**)

A aproximação $\Delta \ln y \approx \Delta \frac{y}{y}$ vale para variações pequenas; para variações grandes, usa-se $e^{\beta} - 1$. O logaritmo também **comprime a assimetria** de variáveis como renda e preços e tende a estabilizar a variância, o que aproxima os dados das premissas do OLS.

Capítulo 11 — Transformações Logarítmicas e Elasticidades

11.1. Por que usar logaritmo

Três motivos práticos levam economistas a trabalhar frequentemente com o log de variáveis monetárias (PIB, salário, preços): (1) variáveis econômicas costumam ter distribuição assimétrica à direita (poucos valores muito altos, muitos valores baixos/moderados) — o log aproxima essa distribuição de uma normal, ajudando a satisfazer a premissa de normalidade dos erros; (2) o log estabiliza a variância, atenuando heterocedasticidade (Capítulo 9) quando ela decorre da escala da variável; e (3), o motivo mais citado na prática, o log transforma **variações absolutas em variações percentuais**, o que confere aos coeficientes uma interpretação de **elasticidade** — a métrica preferida em teoria econômica.

11.2. As quatro combinações log/nível e suas interpretações

Especificação	Modelo	Interpretação de β_1
Linear-linear (nível-nível)	$y = \beta_0 + \beta_1 x + u$	Uma unidade a mais em x está associada a β_1 unidades a mais em y
Log-linear (variável dependente em log)	$\ln(y) = \beta_0 + \beta_1 x + u$	Uma unidade a mais em x está associada a uma variação de $(\beta_1 \times 100)\%$ em y (**semi-elasticidade**)
Linear-log (variável explicativa em log)	$y = \beta_0 + \beta_1 \ln(x) + u$	Um aumento de 1% em x está associado a uma variação de $(\beta_1/100)$ unidades em y
Log-log	$\ln(y) = \beta_0 + \beta_1 \ln(x) + u$	Um aumento de 1% em x está associado a uma variação de $\beta_1\%$ em y (**elasticidade constante**)

A forma **log-log** é a mais usada em economia sempre que o parâmetro de interesse teórico já é, por definição, uma elasticidade — elasticidade-preço da demanda, elasticidade-renda, elasticidade-substituição entre insumos — porque, nesse caso, $\hat{\beta}_1$ **é diretamente** a elasticidade estimada, sem necessidade de conversão.

11.3. Exemplo clássico: elasticidade-renda da demanda (Curva de Engel)

A **Curva de Engel** descreve como o consumo de um bem varia com a renda. Estimando-a em log-log, o coeficiente da renda é diretamente a **elasticidade-renda da demanda**:

$$\ln(\text{quantidade demandada}) = \beta_0 + \beta_1 \ln(\text{renda}) + u$$

```
np.random.seed(3)
n = 200
renda_domicilio = np.random.lognormal(8.5, 0.6, n)
# elasticidade-renda de 0.6 (bem normal, mas "necessidade" -- cresce menos que renda)
qtd_alimento = 2 * renda_domicilio**0.6 * np.random.lognormal(0, 0.15, n)

df_engel = pd.DataFrame({"renda": renda_domicilio, "qtd": qtd_alimento})
df_engel["log_renda"] = np.log(df_engel["renda"])
df_engel["log_qtd"] = np.log(df_engel["qtd"])

modelo_engel = smf.ols("log_qtd ~ log_renda", data=df_engel).fit()
print(f"Elasticidade-renda estimada: {modelo_engel.params['log_renda']:.3f}")
```

A classificação econômica do bem depende diretamente dessa elasticidade estimada:

Elasticidade-renda (β_1)	Classificação do bem
$\beta_1 < 0$	Bem inferior (consumo cai quando a renda sobe)
$0 < \beta_1 < 1$	Bem normal, necessidade (cresce mais devagar que a renda)
$\beta_1 > 1$	Bem normal, de luxo (cresce mais rápido que a renda)

11.4. Cuidado com valores zero ou negativos

$\ln(x)$ não é definido para $x \leq 0$. Isso é um problema prático recorrente com dados econômicos reais: lucro pode ser negativo, algumas observações de gasto podem ser exatamente zero. Não existe solução universalmente aceita — algumas práticas comuns (cada uma com suas próprias críticas na literatura) são usar $\ln(x+1)$ quando $x \geq 0$ pode ser zero, ou restringir a amostra às observações estritamente positivas quando isso é economicamente justificável, documentando sempre a escolha feita.

11.5. Exemplo Resolvido — Elasticidade-preço da demanda

Enunciado: dados simulados de preço e quantidade demandada de um bem, estime a elasticidade-preço da demanda em log-log e classifique a demanda como elástica ou inelástica.

```
np.random.seed(11)
n = 150
preco = np.random.uniform(2, 20, n)
qtd_demandada = 500 * preco**(-1.3) * np.random.lognormal(0, 0.1, n)
```

Solução comentada:

```
df_demanda = pd.DataFrame({"preco": preco, "qtd": qtd_demandada})
df_demanda["log_preco"] = np.log(df_demanda["preco"])
df_demanda["log_qtd"] = np.log(df_demanda["qtd"])

modelo_demanda = smf.ols("log_qtd ~ log_preco", data=df_demanda).fit()
elasticidade_preco = modelo_demanda.params["log_preco"]

print(f"Elasticidade-preço da demanda: {elasticidade_preco:.3f}")
if abs(elasticidade_preco) > 1:
    print("Demanda ELÁSTICA (variação percentual na quantidade > variação no preço)")
else:
    print("Demanda INELÁSTICA (variação percentual na quantidade < variação no preço)")
```

A elasticidade estimada deve ficar próxima de $-1,3$ (o valor usado para simular os dados), confirmando demanda **elástica**: um aumento de 1% no preço reduz a quantidade demandada em aproximadamente "1,3"%. Isso tem implicação direta de política de precificação: para um bem com demanda elástica, um aumento de preço reduz a **receita total** ($P \times Q$), porque a queda percentual em Q supera o aumento percentual em P — o tipo de conclusão que só emerge naturalmente de uma especificação log-log, e que exigiria cálculo adicional numa especificação em nível.

Aprofundando a teoria — o efeito da autocorrelação: tamanho do teste e a estatística de Durbin-Watson
Em séries temporais, o erro de hoje costuma se parecer com o de ontem: $\varepsilon_t = \rho\varepsilon_{t-1} + u_t$. O OLS continua não viesado (se a exogeneidade vale), mas o erro-padrão usual **subestima** a incerteza quando $\rho > 0$: testes rejeitam hipóteses verdadeiras bem mais que os 5% nominais. A estatística de Durbin-Watson satisfaz $\text{DW} \approx 2(1-\hat{\rho})$: perto de 2 indica ausência de autocorrelação de primeira ordem; perto de 0, autocorrelação positiva. Simulação: $H_0 : \beta_1 = 0$ **verdadeira**, com erros AR(1) e regressor persistente; contamos quantas vezes o teste a 5% rejeita:

```
from statsmodels.stats.stattools import durbin_watson
rng_a = np.random.default_rng(31)
n_a, reps_a, rho_a = 120, 1500, 0.7
rej_ols = rej_hac = 0
dw_lista = []
for _ in range(reps_a):
    x = np.zeros(n_a); e = np.zeros(n_a)
    for t in range(1, n_a):
        x[t] = 0.8 * x[t - 1] + rng_a.normal()                    # regressor
persistente
        e[t] = rho_a * e[t - 1] + rng_a.normal()                   # erro AR(1)
    y = 0.0 * x + e                                                 # efeito verdadeiro
NULO
    ajuste = sm.OLS(y, sm.add_constant(x)).fit()
    rej_ols += ajuste.pvalues[1] < 0.05
    rej_hac += ajuste.get_robustcov_results(cov_type="HAC", maxlags=6).pvalues[1] < 0.05
    dw_lista.append(durbin_watson(ajuste.resid))
print(f"Rejeição de H0 (verdadeira) a 5%: OLS clássico = {rej_ols / reps_a:.1%} | Newey-
West (HAC) = {rej_hac / reps_a:.1%}")
print(f"Durbin-Watson médio = {np.mean(dw_lista):.2f} (teórico ≈ 2(1 - {rho_a}) = {2 * (1 -
rho_a):.2f})")
```

```
Rejeição de H0 (verdadeira) a 5%: OLS clássico = 28.9% | Newey-West (HAC) = 13.1%
Durbin-Watson médio = 0.68 (teórico ≈ 2(1 - 0.7) = 0.60)
```

Intuição
O teste "de 5%" do OLS clássico rejeita uma hipótese **verdadeira** em cerca de 29% das amostras: a autocorrelação faz o erro-padrão subestimar a incerteza real. O Newey-West reduz bastante a distorção (para cerca de 13%), mas **não a elimina** em amostras pequenas com erros muito persistentes — por isso, em séries temporais, convém também modelar a dinâmica (incluir defasagens) em vez de só corrigir o erro-padrão.

Capítulo 12 — Autocorrelação e a Curva de Phillips

12.1. O que é autocorrelação e por que é comum em séries temporais

Autocorrelação (ou correlação serial) ocorre quando o erro de um período está correlacionado com o erro de período(s) anterior(es): $\text{Cov}(u_t, u_{t-1}) \neq 0$. É a violação, em contexto de séries temporais, de uma premissa análoga à premissa 5 (Capítulo 3), e é **extremamente comum** em dados macroeconômicos — choques que afetam a economia (uma crise, uma mudança de política monetária) tipicamente têm efeitos que persistem por vários períodos, não apenas no período em que ocorrem, gerando erros correlacionados ao longo do tempo mesmo quando o modelo está corretamente especificado.

Assim como a heterocedasticidade, a autocorrelação não viesa $\hat{\beta}$, mas invalida os erros-padrão usuais do OLS — tipicamente **subestimando-os**, o que faz variáveis parecerem mais significativas do que realmente são (o oposto do padrão mais comum de heterocedasticidade).

12.2. Testando autocorrelação: Durbin-Watson

O teste de **Durbin-Watson** é o mais tradicional, definido como:

$$\text{DW} = \frac{\sum_{t=2}^{n}(\hat{u}_t - \hat{u}_{t-1})^2}{\sum_{t=1}^{n}\hat{u}_t^2}$$

```python
from statsmodels.stats.stattools import durbin_watson

dw = durbin_watson(modelo.resid)
print(f"Durbin-Watson: {dw:.4f}")
```

Valor de DW	Interpretação
Próximo de 2	Sem evidência de autocorrelação
Próximo de 0 (< "1,5", regra prática)	Autocorrelação positiva forte (erro de hoje parecido com o de ontem)
Próximo de 4 (> "2,5", regra prática)	Autocorrelação negativa (erro de hoje tende a ser oposto ao de ontem)

O teste de Durbin-Watson tem uma limitação relevante: só testa autocorrelação de **primeira ordem** (defasagem de 1 período) e exige que o modelo tenha intercepto e não tenha a variável dependente defasada como regressor. Para defasagens múltiplas, usa-se o teste de **Breusch-Godfrey**, mais geral.

12.3. Correção: erros-padrão de Newey-West

Análogo aos erros-padrão robustos a heterocedasticidade (Capítulo 9.4), existem erros-padrão robustos a autocorrelação (e heterocedasticidade simultaneamente) — os erros-padrão de **Newey-West** (HAC — *Heteroskedasticity and Autocorrelation Consistent*):

```
modelo_hac = sm.OLS(y, X).fit(cov_type="HAC", cov_kwds={"maxlags": 2})
print(modelo_hac.summary())
```

O parâmetro `maxlags` define até que defasagem a autocorrelação é considerada na correção — uma escolha que, na prática, costuma seguir a regra empírica maxlags $\approx 0,75 \cdot n^{1/3}$ (Newey e West, 1994) ou ser fixada com base na periodicidade dos dados (por exemplo, 12 para dados mensais, quando se espera sazonalidade residual nos erros).

12.4. Exemplo Resolvido — A Curva de Phillips completa, com diagnóstico

Enunciado: estime a Curva de Phillips com dados reais do BCB (IPCA contra taxa de desemprego e IPCA defasado), teste autocorrelação nos resíduos, e reporte o modelo com a correção apropriada se necessário.

Solução comentada:

```
ipca = sgs.get({"ipca": 433}, start="2012-01-01")
desemprego = sgs.get({"desemprego": 24369}, start="2012-01-01")

df = pd.merge(ipca, desemprego, on="Date")
df["ipca_lag1"] = df["ipca"].shift(1)
df = df.dropna()

X = sm.add_constant(df[["desemprego", "ipca_lag1"]])
y = df["ipca"]
modelo = sm.OLS(y, X).fit()

dw = durbin_watson(modelo.resid)
print(f"Durbin-Watson: {dw:.4f}")

if dw < 1.5 or dw > 2.5:
    print("Evidência de autocorrelação — recorrendo a erros-padrão HAC")
    modelo_final = sm.OLS(y, X).fit(cov_type="HAC", cov_kwds={"maxlags": 4})
else:
    print("Sem evidência forte de autocorrelação — erros-padrão usuais são razoáveis")
    modelo_final = modelo

print(modelo_final.summary())
print(f"\nCoeficiente desemprego: {modelo_final.params['desemprego']:.4f}")
print(f"P-valor (com correção, se aplicável): {modelo_final.pvalues['desemprego']:.4f}")
```

Dados de inflação mensal frequentemente exibem `Durbin-Watson` abaixo de "1,5", refletindo a persistência natural de choques inflacionários (um choque de custos, por exemplo, tende a se propagar por vários meses via reajustes de contratos e expectativas). Isso reforça por que a inclusão de `ipca_lag1` como regressor já é, em parte, uma tentativa de capturar essa dinâmica dentro do próprio modelo — mas mesmo assim é prudente checar se ainda resta autocorrelação residual não capturada, e reportar erros-padrão HAC quando ela persistir, em vez de confiar cegamente nos erros-padrão do OLS simples.

> **Aprofundando a teoria — efeitos fixos por dentro: a transformação "within" é igual às dummies de unidade**
> No modelo $y_{it} = \alpha_i + \beta x_{it} + \varepsilon_{it}$, o efeito específico α_i (cultura, geografia, qualidade das instituições) é *constante no tempo* e, se correlacionado com x, enviesa o OLS agrupado. O estimador de **efeitos fixos** elimina α_i subtraindo de cada variável a média da própria unidade (transformação *within*): $y_{it} - |y|_i = \beta(x_{it} - |x|_i) + (\varepsilon_{it} - |\varepsilon|_i)$. É numericamente idêntico a incluir uma dummy para cada unidade (LSDV). A simulação cria um painel onde **países mais ricos em α_i têm maior x**, e o verdadeiro efeito é $\beta = 1$:

```
import statsmodels.formula.api as smf
rng_pn = np.random.default_rng(40)
N, T = 40, 8
```

```
alfa = rng_pn.normal(0, 3, N)                                    # efeito fixo de cada
país
dados_pn = pd.DataFrame({"pais": np.repeat(np.arange(N), T), "ano": np.tile(np.arange(T),
N)})
dados_pn["x"] = alfa[dados_pn["pais"]] + rng_pn.normal(0, 1, N * T)    # x correlacionado
com o efeito fixo
dados_pn["y"] = alfa[dados_pn["pais"]] + 1.0 * dados_pn["x"] + rng_pn.normal(0, 1, N * T)

pooled = smf.ols("y ~ x", dados_pn).fit()
lsdv = smf.ols("y ~ x + C(pais)", dados_pn).fit()
dentro = dados_pn.assign(**{c: dados_pn[c] - dados_pn.groupby("pais")[c].transform("mean")
for c in ("x", "y")})
within = smf.ols("y ~ x - 1", dentro).fit()
print(f"Verdadeiro: beta = 1.0 | OLS agrupado (pooled): {pooled.params['x']:.3f}
(enviesado) | LSDV: {lsdv.params['x']:.3f} | within: {within.params['x']:.3f}")
print(f"Pooled R² = {pooled.rsquared:.3f}, mas viesado: o x absorve o efeito fixo omitido
(correlação x-alfa = {np.corrcoef(dados_pn['x'], alfa[dados_pn['pais']])[0, 1]:.2f})")
```

```
Verdadeiro: beta = 1.0 | OLS agrupado (pooled): 1.840 (enviesado) | LSDV: 0.997 | within:
0.997
Pooled R² = 0.928, mas viesado: o x absorve o efeito fixo omitido (correlação x-alfa =
0.92)
```

Capítulo 13 — Dados em Painel

13.1. O que é um painel e por que ele resolve parte do problema de exogeneidade

Dados em **painel** combinam a dimensão de corte transversal (várias unidades — países, empresas, indivíduos) com a dimensão temporal (várias observações ao longo do tempo para cada unidade). A grande vantagem sobre um corte transversal puro é permitir controlar por características **não observadas, mas fixas ao longo do tempo**, de cada unidade — os chamados **efeitos fixos individuais**, um instrumento poderoso contra o viés de variável omitida do Capítulo 7, na medida em que a variável omitida seja constante no tempo (ex: cultura institucional de um país, "qualidade de gestão" não mensurada de uma empresa).

```
np.random.seed(42)
anos = range(2010, 2025)
paises = ["Brasil", "Chile", "Colômbia", "Peru", "México"]

dados = []
for pais in paises:
    efeito_fixo = np.random.normal(0, 1)  # característica não observada, fixa no tempo
    for ano in anos:
        pib = 2.0 + efeito_fixo + 0.5 * (ano - 2010) + np.random.normal(0, 1)
        inflacao = np.random.gamma(2, 2) + efeito_fixo
        dados.append({"pais": pais, "ano": ano, "pib": pib, "inflacao": inflacao})

df_painel = pd.DataFrame(dados)
```

13.2. Pooled OLS vs. Efeitos Fixos

O **Pooled OLS** simplesmente ignora a estrutura de painel, tratando cada observação (país-ano) como independente:

```
X = sm.add_constant(df_painel[["inflacao"]])
y = df_painel["pib"]
pooled = sm.OLS(y, X).fit()
```

```
print("POOLED OLS")
print(pooled.summary())
```

O problema: se `efeito_fixo` (não observado) está correlacionado com `inflacao` — como está, por construção, neste exemplo simulado — o Pooled OLS sofre exatamente o viés de variável omitida do Capítulo 7.2. O modelo de **Efeitos Fixos** resolve isso introduzindo uma dummy para cada unidade (país), absorvendo qualquer característica não observada que seja **constante no tempo** para aquela unidade:

```
X_fe = sm.add_constant(pd.get_dummies(df_painel[["inflacao", "pais"]], columns=["pais"],
drop_first=True))
y = df_painel["pib"]
fe = sm.OLS(y, X_fe).fit()
print("\nEFEITOS FIXOS (via dummies)")
print(fe.summary())
```

Na prática, a biblioteca `linearmodels` oferece uma sintaxe mais direta e eficiente (sem precisar criar manualmente uma dummy por país):

```
from linearmodels.panel import PanelOLS, RandomEffects

df_idx = df_painel.set_index(["pais", "ano"])

fe = PanelOLS.from_formula("pib ~ inflacao + EntityEffects", data=df_idx)
print(fe.fit())

re = RandomEffects.from_formula("pib ~ inflacao", data=df_idx)
print(re.fit())
```

13.3. Efeitos fixos vs. efeitos aleatórios: o Teste de Hausman

O modelo de **Efeitos Aleatórios** trata o efeito específico da unidade como parte do termo de erro (aleatório, não correlacionado com os regressores) — é mais eficiente (erros-padrão menores) que efeitos fixos **se** essa suposição de não correlação for válida, mas é **inconsistente** (viesado mesmo com amostra infinita) se ela não for. O **Teste de Hausman** compara os dois formalmente:

$$H_0 : \text{efeitos aleatórios são consistentes (não há correlação entre efeito individual e regressores)}$$

```
from linearmodels.panel import compare
print(compare({"FE": fe.fit(), "RE": re.fit()}))
```

Regra prática de decisão: se o teste de Hausman **rejeita** H_0 (p-valor baixo), use efeitos fixos — há evidência de que o efeito específico da unidade está correlacionado com os regressores, e efeitos aleatórios estaria viesado. Se **não rejeita**, efeitos aleatórios é preferível por ser mais eficiente (aproveita melhor a variação entre unidades, não apenas dentro de cada unidade ao longo do tempo).

13.4. Exemplo Resolvido — Comparando Pooled, FE e RE na prática

Enunciado: usando os dados simulados de `df_painel` (onde, por construção, `efeito_fixo` está correlacionado com `inflacao`), compare o coeficiente de `inflacao` nos três modelos (Pooled, FE, RE) e discuta qual deveria ser mais confiável neste caso específico.

Solução comentada:

```
X_pooled = sm.add_constant(df_painel["inflacao"])
pooled = sm.OLS(df_painel["pib"], X_pooled).fit()

df_idx = df_painel.set_index(["pais", "ano"])
fe_res = PanelOLS.from_formula("pib ~ inflacao + EntityEffects", data=df_idx).fit()
```

```
re_res = RandomEffects.from_formula("pib ~ inflacao", data=df_idx).fit()

print(f"Pooled OLS: {pooled.params['inflacao']:.4f}")
print(f"Efeitos Fixos: {fe_res.params['inflacao']:.4f}")
print(f"Efeitos Aleatórios: {re_res.params['inflacao']:.4f}")
```

Como, por construção, `efeito_fixo` afeta tanto `pib` quanto `inflacao` (ambos somam `efeito_fixo` na simulação), o Pooled OLS tende a produzir um coeficiente de `inflacao` **viesado** — capturando parte do efeito de `efeito_fixo`, que fica escondido no termo de erro. O modelo de Efeitos Fixos, ao introduzir uma dummy por país, absorve exatamente essa fonte de viés (qualquer característica constante do país, observada ou não, é removida da comparação), tornando o coeficiente de `inflacao` estimado **dentro de cada país ao longo do tempo** — mais próximo do verdadeiro efeito causal simulado, neste caso controlado. O modelo de Efeitos Aleatórios, por assumir (incorretamente, neste exemplo) que o efeito individual não é correlacionado com `inflacao`, tende a ficar mais próximo do Pooled OLS viesado do que do Efeitos Fixos — exatamente o cenário em que o Teste de Hausman rejeitaria a adequação de efeitos aleatórios.

> **Aprofundando a teoria — a regressão espúria de Granger e Newbold**
> Duas séries **independentes** com raiz unitária (passeios aleatórios) costumam produzir, ao serem regredidas uma na outra, R^2 alto e estatísticas t "significativas" — um resultado puramente *espúrio*, porque os erros herdam a não estacionariedade e a inferência padrão deixa de valer (Granger e Newbold, 1974). É a razão central para testar estacionariedade (ADF) antes de regredir séries em níveis, e para trabalhar com diferenças (ou testar cointegração). A simulação regride 1.000 pares de passeios aleatórios **independentes**:

```
from statsmodels.tsa.stattools import adfuller
rng_r = np.random.default_rng(50)
n_r, reps_r = 200, 1000
sig_niveis = sig_difs = 0
r2_niveis = []
for _ in range(reps_r):
    a = np.cumsum(rng_r.normal(size=n_r)); b = np.cumsum(rng_r.normal(size=n_r))    #
passeios aleatórios independentes
    ajuste_n = sm.OLS(a, sm.add_constant(b)).fit()
    ajuste_d = sm.OLS(np.diff(a), sm.add_constant(np.diff(b))).fit()
    sig_niveis += ajuste_n.pvalues[1] < 0.05
    sig_difs += ajuste_d.pvalues[1] < 0.05
    r2_niveis.append(ajuste_n.rsquared)
print(f"Regressão em NÍVEIS: 'significativa' a 5% em {sig_niveis / reps_r:.0%} dos pares
(esperado: 5%) | R² médio = {np.mean(r2_niveis):.2f}")
print(f"Regressão em DIFERENÇAS: 'significativa' a 5% em {sig_difs / reps_r:.1%} dos
pares")

caminho = np.cumsum(rng_r.normal(size=300))
print(f"ADF do passeio aleatório: p = {adfuller(caminho)[1]:.2f} (não rejeita raiz
unitária) | ADF da série diferenciada: p = {adfuller(np.diff(caminho))[1]:.4f}")
```

```
Regressão em NÍVEIS: 'significativa' a 5% em 85% dos pares (esperado: 5%) | R² médio = 0.25
Regressão em DIFERENÇAS: 'significativa' a 5% em 5.0% dos pares
ADF do passeio aleatório: p = 0.76 (não rejeita raiz unitária) | ADF da série diferenciada:
p = 0.0000
```

Capítulo 14 — Séries Temporais: Estacionariedade

14.1. Por que estacionariedade importa

Uma série temporal é **estacionária** (em sentido fraco) quando sua média, variância e estrutura de autocorrelação não mudam ao longo do tempo. A maior parte da teoria de séries temporais (incluindo tudo o que vem no Capítulo 15) pressupõe estacionariedade — regredir uma série não estacionária contra outra não estacionária pode produzir o fenômeno da **regressão espúria**: R^2 e estatísticas t enganosamente altos entre duas séries que não têm nenhuma relação causal real, apenas porque ambas têm tendência.

O exemplo didático clássico (Granger e Newbold, 1974): regredir o PIB do Brasil contra a população da Índia ao longo do tempo produziria, quase certamente, um coeficiente "significativo" — porque ambas as séries **crescem** ao longo do tempo (tendência), não porque exista qualquer relação causal.

14.2. Raiz unitária e passeio aleatório

O caso canônico de série não estacionária é o **passeio aleatório** (*random walk*): $y_t = y_{t-1} + \varepsilon_t$. Séries de preços de ativos financeiros e, em boa aproximação, muitas séries macroeconômicas em nível (PIB, índice de preços) se comportam como passeios aleatórios ou próximo disso — tecnicamente, dizem ter uma **raiz unitária**.

14.3. O teste ADF (Augmented Dickey-Fuller)

O teste mais usado para diagnosticar raiz unitária é o **ADF**:

$$H_0 : \text{a série tem raiz unitária (não é estacionária)} \quad \text{vs.} \quad H_1 : \text{a série é estacionária}$$

Repare que a hipótese nula do ADF é o **oposto** do padrão usual em outros testes deste capítulo (onde H_0 costuma ser "nenhum problema") — aqui, H_0 é "a série TEM o problema" (raiz unitária). Isso significa que **não rejeitar** H_0 (p-valor alto) é a má notícia (série não estacionária), e **rejeitar** H_0 (p-valor baixo) é a boa notícia (série estacionária) — o inverso do que a intuição de "p baixo é ruim" sugeriria em outros contextos, e uma fonte comum de confusão.

```
from statsmodels.tsa.stattools import adfuller

ipca = sgs.get({"ipca": 433}, start="2000-01-01")
result = adfuller(ipca["ipca"].dropna())

print(f"ADF Statistic: {result[0]:.4f}")
print(f"p-valor: {result[1]:.4f}")
print("Valores críticos:")
for chave, valor in result[4].items():
    print(f"  {chave}: {valor:.4f}")

# Se p-valor < 0.05: rejeita H0 -> série é estacionária
```

14.4. O que fazer com uma série não estacionária: diferenciação

Quando o ADF não rejeita H_0 (série não estacionária), a correção mais comum é **diferenciar** a série — trabalhar com $\Delta y_t = y_t - y_{t-1}$ em vez de y_t em nível. Séries que se tornam estacionárias após uma diferenciação são chamadas **integradas de ordem 1**, denotadas $I(1)$; séries já estacionárias em nível são $I(0)$. Esse "d" (grau de diferenciação necessário) é exatamente o parâmetro d do modelo ARIMA(p, d, q), assunto do próximo capítulo.

```
ipca_diff = ipca["ipca"].diff().dropna()
result_diff = adfuller(ipca_diff)
print(f"ADF (série diferenciada) p-valor: {result_diff[1]:.4f}")
```

14.5. Exemplo Resolvido — Diagnosticando estacionariedade da Selic

Enunciado: teste se a série da taxa Selic (nível) é estacionária. Se não for, diferencie e teste novamente. Interprete o resultado economicamente.

Solução comentada:

```
selic = sgs.get({"selic": 11}, start="2000-01-01").dropna()

result_nivel = adfuller(selic["selic"])
print(f"Selic (nível) — p-valor ADF: {result_nivel[1]:.4f}")

if result_nivel[1] > 0.05:
    print("Não estacionária em nível — diferenciando...")
    selic_diff = selic["selic"].diff().dropna()
    result_diff = adfuller(selic_diff)
    print(f"Selic (diferenciada) — p-valor ADF: {result_diff[1]:.4f}")
```

É comum que a Selic em **nível** não rejeite a hipótese de raiz unitária — faz sentido econômico, já que a taxa de juros passa longos períodos em patamares específicos definidos pela política monetária, sem reverter rapidamente a uma média fixa de longo prazo (o Banco Central muda a meta e a série "persiste" no novo patamar por vários meses ou anos, um comportamento consistente com não estacionariedade em amostras finitas). Já a série **diferenciada** (a variação da Selic reunião a reunião) tende a ser estacionária, porque reflete decisões de ajuste que não se acumulam indefinidamente na mesma direção — o Copom sobe e desce a taxa ao longo dos ciclos, então Δ Selic oscila em torno de zero, sem tendência de longo prazo.

Exemplo resolvido — simulando um AR(1), recuperando o parâmetro e separando treino de teste
Gera-se uma série AR(1), $y_t = 0,7y_{t-1} + \varepsilon_t$, ajusta-se um ARIMA("1,0",0) e compara-se a previsão com a de um modelo ingênuo ("amanhã igual a hoje"). Em séries temporais, a avaliação **respeita a ordem do tempo**: treina-se no passado e testa-se no futuro (embaralhar as observações vaza informação do futuro para o treino).

```
from statsmodels.tsa.arima.model import ARIMA
rng_ar = np.random.default_rng(60)
n_ar = 520
y_ar = np.zeros(n_ar)
for t in range(1, n_ar):
    y_ar[t] = 0.7 * y_ar[t - 1] + rng_ar.normal()
n_treino = 300
mod = ARIMA(y_ar[:n_treino], order=(1, 0, 0)).fit()                          # parâmetros
estimados SÓ com o passado
um_passo = mod.apply(y_ar).get_prediction(start=n_treino).predicted_mean     # previsões 1
passo à frente no período de teste
real = y_ar[n_treino:]
ingenuo = y_ar[n_treino - 1:-1]                                               # "amanhã
igual a hoje"
print(f"phi estimado: {mod.params[1]:.3f} (verdadeiro 0.700) | erro-padrão:
{mod.bse[1]:.3f} | AIC: {mod.aic:.1f}")
print(f"{len(real)} previsões fora da amostra | RMSE do ARIMA(1,0,0):
{np.sqrt(np.mean((real - um_passo) ** 2)):.3f} | RMSE do ingênuo: {np.sqrt(np.mean((real -
ingenuo) ** 2)):.3f}")
prev = mod.get_forecast(steps=10)
ic = prev.conf_int(alpha=0.05)
print(f"Previsão a 10 passos: {prev.predicted_mean[-1]:.2f} (a série tende à média, 0) com
IC 95% = ({ic[-1, 0]:.2f}, {ic[-1, 1]:.2f})")
```

```
phi estimado: 0.647 (verdadeiro 0.700) | erro-padrão: 0.048 | AIC: 837.3
220 previsões fora da amostra | RMSE do ARIMA(1,0,0): 0.943 | RMSE do ingênuo: 0.977
Previsão a 10 passos: 0.41 (a série tende à média, 0) com IC 95% = (-2.07, 2.89)
```

Intuição
O ARIMA tem erro de previsão menor que o ingênuo porque *usa a estrutura* da série (reversão à média com $\varphi = 0,7$): o ingênuo carrega o ruído de hoje inteiro para amanhã. A diferença é modesta (a série é muito ruidosa), e em alguns trechos curtos o ingênuo pode ganhar por acaso — um motivo para avaliar com **muitas** previsões fora da amostra, e não com poucas. Note também que o intervalo de confiança a **10 passos** é muito mais largo: a incerteza cresce com o horizonte.

Atenção — erro comum
Um modelo ARIMA previsor **não explica causas**: ele extrapola a estrutura temporal passada. Um bom ajuste in-sample (AIC baixo) não garante boa previsão fora da amostra; compare sempre com um *benchmark simples* (o modelo ingênuo, a média móvel). A causalidade de Granger, por sua vez, é um conceito de *precedência preditiva* ("os valores passados de x ajudam a prever y?"), não de causalidade em sentido estrutural.

Capítulo 15 — ARIMA, Cointegração, VAR e Causalidade de Granger

15.1. Modelos ARIMA

Um modelo **ARIMA(p, d, q)** combina três componentes: **AR** (autorregressivo, de ordem p — a série depende de seus próprios valores passados), **I** (integrado, de ordem d — quantas diferenciações são necessárias para estacionariedade, Capítulo 14.4) e **MA** (médias móveis, de ordem q — a série depende de erros passados).

$$y_t = c + \sum_{i=1}^{p} \phi_i y_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t$$

(equação escrita para a série já diferenciada d vezes, se $d > 0$).

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

ipca = sgs.get({"ipca": 433}, start="2000-01-01")

# ACF e PACF ajudam a identificar p e q "a olho"
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
plot_acf(ipca["ipca"], lags=24, ax=ax1)
plot_pacf(ipca["ipca"], lags=24, ax=ax2)
fig.tight_layout()

modelo_arima = ARIMA(ipca["ipca"], order=(1, 0, 1))
resultado = modelo_arima.fit()
print(resultado.summary())

forecast = resultado.forecast(steps=12)
print("Previsão para os próximos 12 meses:")
print(forecast)
```

A seleção de (p, d, q) pode ser guiada visualmente pelos gráficos ACF/PACF, ou automatizada por busca em grade, escolhendo a combinação com menor **AIC** (Critério de Informação de Akaike, que penaliza modelos com mais parâmetros, análogo em espírito ao R^2 ajustado do Capítulo 4.2):

```
import itertools
```

```
melhor_aic = float("inf")
melhor_ordem = None
for p, d, q in itertools.product(range(4), range(2), range(4)):
    try:
        res = ARIMA(ipca["ipca"], order=(p, d, q)).fit()
        if res.aic < melhor_aic:
            melhor_aic, melhor_ordem = res.aic, (p, d, q)
    except Exception:
        continue

print(f"Melhor ARIMA{melhor_ordem} com AIC = {melhor_aic:.2f}")
```

15.2. Cointegração: quando duas séries não estacionárias "andam juntas"

Duas séries individualmente não estacionárias — como câmbio e Selic, ou preço doméstico e preço internacional de uma commodity — podem, ainda assim, ter uma combinação linear entre elas que **é** estacionária. Quando isso ocorre, diz-se que as séries são **cointegradas**: mesmo se afastando temporariamente, existe uma relação de equilíbrio de longo prazo para a qual tendem a retornar — o que evita o problema de regressão espúria do Capítulo 14.1, apesar de ambas as séries serem $I(1)$.

```
from statsmodels.tsa.stattools import coint

cambio = sgs.get({"cambio": 1}, start="2000-01-01")
selic = sgs.get({"selic": 11}, start="2000-01-01")
df_ci = pd.merge(cambio, selic, on="Date").dropna()

score, pvalue, _ = coint(df_ci.iloc[:, 0], df_ci.iloc[:, 1])
print(f"Teste de Cointegração (Engle-Granger) — p-valor: {pvalue:.4f}")
if pvalue < 0.05:
    print("As séries são cointegradas (relação de longo prazo)")
else:
    print("Sem evidência de cointegração")
```

15.3. Modelos VAR e Função Impulso-Resposta

Um **VAR (Vetor Autorregressivo)** generaliza o AR de uma única série para um sistema de várias séries que se afetam mutuamente, sem impor de antemão qual variável é "causa" e qual é "efeito" — cada variável do sistema é modelada como função de valores passados de si mesma **e** de todas as outras variáveis do sistema:

```
from statsmodels.tsa.api import VAR

pib = sgs.get({"pib": 4380}, start="2005-01-01")
ipca = sgs.get({"ipca": 433}, start="2005-01-01")
selic = sgs.get({"selic": 11}, start="2005-01-01")

df_var = pd.merge(pib, ipca, on="Date")
df_var = pd.merge(df_var, selic, on="Date").dropna()
df_var.columns = ["pib", "ipca", "selic"]

modelo_var = VAR(df_var)
resultado_var = modelo_var.fit(maxlags=6, ic="aic")
print(f"Defasagens ótimas: {resultado_var.k_ar}")
print(resultado_var.summary())

# Função Impulso-Resposta: como um choque em uma variável se propaga às demais
irf = resultado_var.irf(12)
fig = irf.plot()
fig.tight_layout()

forecast = resultado_var.forecast(df_var.values[-resultado_var.k_ar:], steps=6)
```

```
forecast_df = pd.DataFrame(forecast, columns=df_var.columns)
print("Previsão 6 meses:")
print(forecast_df)
```

A **Função Impulso-Resposta** (IRF) responde a uma pergunta central de política econômica: "se a Selic sofrer um choque de um desvio-padrão hoje, como o PIB e o IPCA reagem nos próximos meses?" — é a ferramenta padrão para simular o efeito dinâmico de um choque de política monetária num sistema macroeconômico, sem precisar especificar a priori a direção da causalidade entre as variáveis.

15.4. Causalidade de Granger

Apesar do nome, a **causalidade de Granger** não testa causalidade no sentido filosófico/estrutural do Capítulo 7 — testa uma noção mais restrita e puramente preditiva: "os valores passados de x ajudam a prever y, além do que os valores passados do próprio y já prevêem?". Se sim, diz-se que "x Granger-causa y".

```
from statsmodels.tsa.stattools import grangercausalitytests

gc = grangercausalitytests(df_var[["ipca", "selic"]], maxlag=6, verbose=False)

for lag in range(1, 7):
    p = gc[lag][0]["ssr_ftest"][1]
    print(f"Lag {lag}: p-valor = {p:.4f} {'Selic Granger-causa IPCA' if p < 0.05 else 'Não
causa (Granger)'}")
```

Gotcha importante: causalidade de Granger é sobre **precedência temporal e poder preditivo**, não sobre mecanismo causal estrutural. É inteiramente possível que x Granger-cause y sem que exista qualquer relação causal genuína — por exemplo, se ambas respondem, com defasagens diferentes, a um terceiro fator comum (voltando ao tema central do Capítulo 7). O teste é uma ferramenta útil e amplamente usada, mas seu nome é, reconhecidamente, uma fonte perene de mal-entendidos — o próprio Clive Granger, ao propor o teste em 1969, foi cuidadoso em ressaltar essa limitação.

15.5. Exemplo Resolvido — Selic causa IPCA no sentido de Granger?

Enunciado: usando os dados de `df_var` do item 15.3, teste se a Selic Granger-causa o IPCA e, separadamente, se o IPCA Granger-causa a Selic. Interprete o resultado à luz da lógica de transmissão de política monetária (a Selic deveria afetar a inflação com defasagem, dado o mecanismo de transmissão da política monetária).

Solução comentada:

```
print("SELIC -> IPCA")
gc_selic_ipca = grangercausalitytests(df_var[["ipca", "selic"]], maxlag=6, verbose=False)
for lag in range(1, 7):
    p = gc_selic_ipca[lag][0]["ssr_ftest"][1]
    print(f"  Lag {lag}: p = {p:.4f}")

print("\nIPCA -> SELIC")
gc_ipca_selic = grangercausalitytests(df_var[["selic", "ipca"]], maxlag=6, verbose=False)
for lag in range(1, 7):
    p = gc_ipca_selic[lag][0]["ssr_ftest"][1]
    print(f"  Lag {lag}: p = {p:.4f}")
```

Um resultado tipicamente esperado (e consistente com a teoria de transmissão de política monetária, que prevê defasagens de vários meses até trimestres entre a decisão do Copom e seu efeito pleno sobre a inflação) é encontrar Granger-causalidade de `selic` para `ipca` em defasagens intermediárias (por exemplo, 3 a 6 meses), e possivelmente também de `ipca` para `selic` em defasagens curtas — o que faria sentido, já que o Copom **reage** à inflação corrente e esperada ao definir a Selic (a chamada regra de Taylor, Apostila 5). Encontrar Granger-causalidade nas duas direções não é uma contradição: reflete exatamente a natureza de retroalimentação entre política monetária e inflação, que é a razão pela qual o VAR (que modela as duas variáveis simetricamente, sem

impor uma direção única) é a ferramenta mais adequada para este tipo de sistema — em vez de uma regressão simples de uma variável contra a outra, que precisaria escolher arbitrariamente qual é "a variável dependente".

Problema	Sintoma	Remédio
Forma funcional errada	RESET rejeita; padrão curvo nos resíduos	Termos quadráticos, logs, interações
Variável omitida	Coeficiente muda muito ao incluir controles	Incluir controles; identificação (IV, DiD)
Multicolinearidade	VIF alto; erros-padrão grandes	Mais dados, índice, remover redundância
Heterocedasticidade	Breusch-Pagan rejeita; funil nos resíduos	Erros-padrão robustos (HC3)
Autocorrelação	Durbin-Watson longe de 2	Newey-West; modelar a dinâmica
Raiz unitária	ADF não rejeita; R^2 alto espúrio	Diferenciar; testar cointegração
Efeito não observado constante	Pooled difere de FE (Hausman)	Efeitos fixos

---

Capítulo 16 — Exercícios para Executar (na mão)

Estes exercícios não exigem computador: o objetivo é treinar a leitura e o raciocínio estatístico sobre resultados já prontos, antes de rodar código. Resolva no papel. **As soluções não estão neste documento** — quando terminar, peça para eu conferir suas respostas.

Exercício 1 — Interpretando um coeficiente log-linear
Uma regressão estima $\ln(\text{salário}) = 6,2 + 0,085 \cdot \text{escolaridade}$. Interprete o coeficiente `"0,085"` em português econômico (use tanto a aproximação simples quanto a fórmula exata de Halvorsen-Palmquist, e compare os dois valores numericamente).

Exercício 2 — Calculando um resíduo manualmente
Um modelo estimado é $\hat{y} = 50 + 2x$. Para uma observação com $x = 30$ e y observado $= 115$, calcule $\hat{y}$, o resíduo $\hat{u}$, e diga se o modelo superestimou ou subestimou o valor observado.

Exercício 3 — Lendo uma saída de regressão fornecida como texto
Dada a saída abaixo (regressão de investimento empresarial contra taxa de juros e crescimento esperado do PIB, $n = 80$), responda: (a) qual variável tem maior significância estatística? (b) o R^2 ajustado sugere um bom ajuste para dados de corte transversal de empresas? (c) o sinal do coeficiente de `juros` é consistente com a teoria econômica de investimento?

```
                coef    std err          t      P>|t|
------------------------------------------------------------------------------
const           12.400      3.100      4.000      0.000
juros           -1.850      0.510     -3.627      0.001
crescimento_pib  0.640      0.290      2.207      0.030
==============================================================================
R-squared: 0.187        Adj. R-squared: 0.166
```

Exercício 4 — Identificando violação de premissa a partir de um gráfico descrito
Um pesquisador plota os resíduos de uma regressão de preço de imóveis contra área construída (eixo x: valores previstos; eixo y: resíduos). Ele descreve o gráfico: "os pontos formam uma curva em U — resíduos positivos para valores previstos baixos e altos, e negativos para valores previstos intermediários". Qual premissa de Gauss-Markov está provavelmente sendo violada, e que mudança na especificação do modelo (não no método de estimação) resolveria o problema?

Exercício 5 — Correlação vs. causalidade num caso concreto
Um estudo encontra correlação positiva e estatisticamente significativa entre "número de bombeiros presentes em um incêndio" e "valor total dos danos causados pelo incêndio", em uma amostra de incêndios urbanos. Um

jornal manchete: “Mais bombeiros causam mais danos”. Explique, usando os conceitos do Capítulo 7 (viés de variável omitida ou causalidade reversa), por que essa conclusão causal é equivocada, identificando explicitamente a variável de confusão mais plausível.

Exercício 6 — Comparando dois intervalos de confiança
Dois modelos estimam o efeito da escolaridade sobre o salário: Modelo A, com $n = 50$, encontra $\hat{\beta} = 0,09$ com IC 95% $[0,01; 0,17]$; Modelo B, com $n = 2000$, encontra $\hat{\beta} = 0,07$ com IC 95% $[0,065; 0,075]$. Qual estimativa você reportaria como mais confiável e por quê? A diferença de magnitude entre os dois pontos estimados ($0,09$ vs. $0,07$) é necessariamente evidência de que um dos dois modelos está “errado”?

Capítulo 17 — Exercícios para Executar (em código)

Implemente e execute cada um no seu editor, com `statsmodels`. **As soluções não estão neste documento** — o objetivo é rodar de verdade, ver a saída, e comparar com sua expectativa teórica; quando terminar, peça para eu revisar seu código.

Exercício 1 — Sua primeira regressão simples
Simule uma série de `renda` (valores de 1000 a 10000, em passos de 500) e uma série de `consumo = 300 + 0.75 * renda + ruído normal(0, 150)`. Rode `sm.OLS` com constante e interprete `const` e o coeficiente de `renda`.

Exercício 2 — Esquecendo o intercepto de propósito
Repita o Exercício 1, mas **sem** usar `sm.add_constant`. Compare o R^2 e o coeficiente de `renda` com o modelo correto. Explique a diferença.

Exercício 3 — Regressão por fórmula vs. por matriz
Rode o mesmo modelo do Exercício 1 usando `smf.ols("consumo ~ renda", data=df).fit()`. Confirme que os coeficientes são idênticos aos obtidos com `sm.OLS`.

Exercício 4 — Intervalos de confiança em diferentes níveis
Usando o modelo do Exercício 1, calcule os intervalos de confiança a 90%, 95% e 99% com `modelo.conf_int(alpha=...)`. Qual fica mais largo? Por quê, do ponto de vista da lógica de “confiança”?

Exercício 5 — Regressão múltipla com três regressores
Baixe (ou simule) dados de `pib`, `inflacao` e `desemprego`. Rode uma regressão de `pib` contra `inflacao` e `desemprego` simultaneamente. Compare o R^2 ajustado desse modelo com o de duas regressões simples separadas.

Exercício 6 — Calculando VIF
Usando os dados do Exercício 5, calcule o VIF de cada variável explicativa com `variance_inflation_factor`. Há sinal de multicolinearidade preocupante?

Exercício 7 — Teste de Breusch-Pagan
Simule dados com heterocedasticidade proposital (erro cuja variância cresce com uma variável `x`, como no Capítulo 9.5) e rode o teste de Breusch-Pagan com `het_breuschpagan`. Confirme que o teste rejeita homocedasticidade.

Exercício 8 — Erros-padrão robustos
Usando o mesmo modelo do Exercício 7, rode a regressão novamente com `cov_type="HC1"` e compare os erros-padrão (`modelo.bse`) com os do modelo original.

Exercício 9 — Durbin-Watson em dados reais
Baixe o IPCA mensal do BCB (`sgs.get({"ipca": 433}, ...)`), regrida-o contra sua própria defasagem de 1 mês, e calcule o Durbin-Watson dos resíduos com `durbin_watson`.

Exercício 10 — Erros-padrão de Newey-West (HAC)
Usando o modelo do Exercício 9, se o Durbin-Watson indicar autocorrelação, reestime com `cov_type="HAC"` e `cov_kwds={"maxlags": 4}`. Compare o p-valor do coeficiente antes e depois da correção.

Exercício 11 — Variável dummy simples
Simule uma variável `salario`, uma variável contínua `experiencia` e uma dummy `setor_publico` (0/1). Construa `salario = 3000 + 200*experiencia + 800*setor_publico + ruído`, rode a regressão e interprete o coeficiente da dummy.

Exercício 12 — Termo de interação
Repita o Exercício 11, mas adicione um termo de interação `experiencia * setor_publico` usando a sintaxe `smf.ols("salario ~ experiencia * setor_publico", data=df)`. O retorno da experiência é o mesmo nos dois setores?

Exercício 13 — Log-linear e a correção de Halvorsen-Palmquist
Usando os dados do Exercício 11, rode a regressão com `np.log(salario)` como variável dependente. Calcule o efeito percentual do setor público sobre o salário usando a aproximação simples e a fórmula exata; compare os dois números.

Exercício 14 — Elasticidade em log-log
Simule `preco` (uniforme entre 5 e 50) e `quantidade = 1000 * preco**(-0.8) * ruído_lognormal`. Rode a regressão em log-log e verifique se a elasticidade estimada recupera o valor `-0.8` usado na simulação. Classifique a demanda como elástica ou inelástica.

Exercício 15 — Teste ADF em duas séries diferentes
Baixe a série da Selic e a série do câmbio (BCB). Rode o teste ADF em nível para as duas. Diferencie cada uma e rode o ADF novamente. Reporte qual(is) série(s) precisou(aram) de diferenciação para ficar estacionária.

Exercício 16 — Teste de cointegração
Usando as séries de Selic e câmbio do Exercício 15 (ambas em nível, mesmo que não estacionárias individualmente), rode o teste de cointegração de Engle-Granger com `coint`. Há evidência de relação de longo prazo entre as duas?

Exercício 17 — Ajustando um ARIMA e prevendo
Baixe o IPCA mensal desde 2010. Ajuste um `ARIMA("1,0",1)`, gere uma previsão de 12 meses com `.forecast(steps=12)`, e plote o histórico dos últimos 24 meses junto com a previsão.

Exercício 18 — Selecionando a melhor ordem ARIMA por AIC
Repita o Exercício 17, mas faça uma busca em grade (`itertools.product`) sobre `p` em `range("0,4")`, `d` em `range("0,2")`, `q` em `range("0,4")`, escolhendo a combinação de menor AIC.

Exercício 19 — Modelo VAR e Função Impulso-Resposta
Monte um DataFrame com PIB, IPCA e Selic mensais (BCB). Estime um `VAR` com `maxlags=6` e `ic="aic"`. Plote a função impulso-resposta (`resultado.irf(12).plot()`) e descreva, em texto, o que acontece com o PIB após um choque na Selic.

Exercício 20 — Causalidade de Granger nas duas direções
Usando o mesmo `df_var` do Exercício 19, teste se a Selic Granger-causa o IPCA e se o IPCA Granger-causa a Selic, para defasagens de 1 a 6 meses. Reporte em qual(is) defasagem(ns) cada relação é significativa a 5%.

Exercício 21 — Painel: Pooled OLS vs. Efeitos Fixos
Monte um painel simulado com 6 países e 15 anos (como no Capítulo 13.1), com um `efeito_fixo` por país correlacionado com a variável explicativa. Rode Pooled OLS e Efeitos Fixos (via dummies ou via `linearmodels.PanelOLS`) e compare os coeficientes.

Exercício 22 — Teste de Hausman
Usando os dados do Exercício 21, rode também o modelo de Efeitos Aleatórios com `RandomEffects` e compare os dois modelos com `linearmodels.panel.compare`. Com base na comparação, qual dos dois modelos (FE ou RE) parece mais adequado?

Capítulo Final — Resumo

Conceito	Código
OLS simples/múltiplo	`sm.OLS(y, sm.add_constant(X)).fit()`
Regressão por fórmula	`smf.ols("y ~ x1 + x2", data=df).fit()`
Coeficientes / p-valores / R^2	`modelo.params` / `modelo.pvalues` / `modelo.rsquared_adj`
Intervalo de confiança	`modelo.conf_int(alpha=0.05)`
VIF (multicolinearidade)	`variance_inflation_factor(X.values, i)`
Teste de Breusch-Pagan	`het_breuschpagan(resid, exog)`
Erros-padrão robustos (HC)	`modelo.fit(cov_type="HC1")`
Durbin-Watson	`durbin_watson(modelo.resid)`
Erros-padrão HAC (Newey-West)	`modelo.fit(cov_type="HAC", cov_kwds={"maxlags": k})`
Dummy / interação	`pd.get_dummies(..., drop_first=True)` / `"y ~ x * dummy"`
Painel — Efeitos Fixos	`PanelOLS.from_formula("y ~ x + EntityEffects", data=df)`
Painel — Efeitos Aleatórios	`RandomEffects.from_formula("y ~ x", data=df)`
Teste de Hausman	`compare({"FE": fe.fit(), "RE": re.fit()})`
Teste ADF (estacionariedade)	`adfuller(serie)`
ARIMA	`ARIMA(y, order=(p,d,q)).fit()`
Cointegração	`coint(y1, y2)`
VAR	`VAR(df).fit(maxlags=6, ic="aic")`
Causalidade de Granger	`grangercausalitytests(df, maxlag=6)`

Os conceitos centrais para carregar desta apostila: um coeficiente de regressão só é confiável na medida em que as premissas de Gauss-Markov (Capítulo 3) se sustentam; R^2 alto não é sinônimo de modelo bem especificado; e — o ponto mais importante de todos — **correlação não é causalidade** (Capítulo 7), a lição que mais separa um uso ingênuo de um uso rigoroso de regressão em economia.

Capítulo 18 — Erros Comuns e Teste Rápido

18.1. Os erros mais comuns em econometria aplicada

#	Erro	Como evitar
1	Interpretar correlação como causalidade	Pergunte pelo mecanismo, confundidores e causalidade reversa
2	Confiar no R^2 alto	Cheque forma funcional e estacionariedade; R^2 mede ajuste, não validade
3	Ignorar o diagnóstico de premissas	Resíduos, RESET, BP, DW, VIF

#	Erro	Como evitar
4	Usar erro-padrão clássico "por padrão"	Robusto (HC3) ou HAC conforme os dados
5	Regredir séries em nível sem testar raiz unitária	ADF; diferenciar ou cointegração
6	Caçar significância (p-hacking)	Pré-especificar; corrigir comparações múltiplas
7	Incluir todas as dummies de uma categoria	$G - 1$ dummies
8	Interpretar um log-linear como se fosse nível	Aproximação $100\beta\%$ ou exata $e^{\beta} - 1$
9	Dizer que um coeficiente não significativo "prova" ausência de efeito	Ausência de evidência não é evidência de ausência (veja o poder)
10	Avaliar previsão no mesmo período em que o modelo foi ajustado	Treino/teste respeitando o tempo

18.2. Teste rápido: verdadeiro ou falso (com gabarito)

1. **V ou F:** Sob H_0 verdadeira, 5% dos p-valores ficam abaixo de "0,05". **Verdadeiro.** É o erro tipo I.
2. **V ou F:** A multicolinearidade viesa os coeficientes. **Falso.** Infla os erros-padrão; não gera viés.
3. **V ou F:** Com heterocedasticidade, o OLS deixa de ser não viesado. **Falso.** Continua não viesado; o erro-padrão usual é que fica errado.
4. **V ou F:** Uma regressão com R^2 de "0,95" entre duas séries em nível prova relação causal. **Falso.** Pode ser espúria (raiz unitária).
5. **V ou F:** Omitir uma variável correlacionada com x e que afeta y viesa o coeficiente de x. **Verdadeiro.**
6. **V ou F:** Em log-log, β é uma elasticidade. **Verdadeiro.**
7. **V ou F:** O Durbin-Watson próximo de 2 indica autocorrelação forte. **Falso.** Indica ausência.
8. **V ou F:** Efeitos fixos eliminam efeitos não observados constantes no tempo. **Verdadeiro.**
9. **V ou F:** Em séries temporais, embaralhar os dados antes de separar treino e teste é uma boa prática. **Falso.** Vaza informação do futuro.
10. **V ou F:** Incluir G dummies e o intercepto é correto. **Falso.** Gera colinearidade perfeita.

Próxima apostila

Com a base de manipulação de dados (Apostila 2), visualização (Apostila 3) e agora econometria (Apostila 4) estabelecida, siga para a **Apostila 5 — Projetos**, onde essas três apostilas se juntam em projetos completos e realistas — dashboards macroeconômicos, testes de convergência de renda, modelos de previsão de inflação e a estrutura de um TCC aplicado — do levantamento de dados brutos até a conclusão escrita.